

BASE DE DONNÉES

ESP3

Njakarison Menja R.

cours BD - R. Njakarison M.



Plan du cours

Partie 1 : Introduction aux bases de données

Partie 2 : Le processus de modélisation

Partie 3 : Le modèle relationnel

Partie 4 : Introduction à SQL

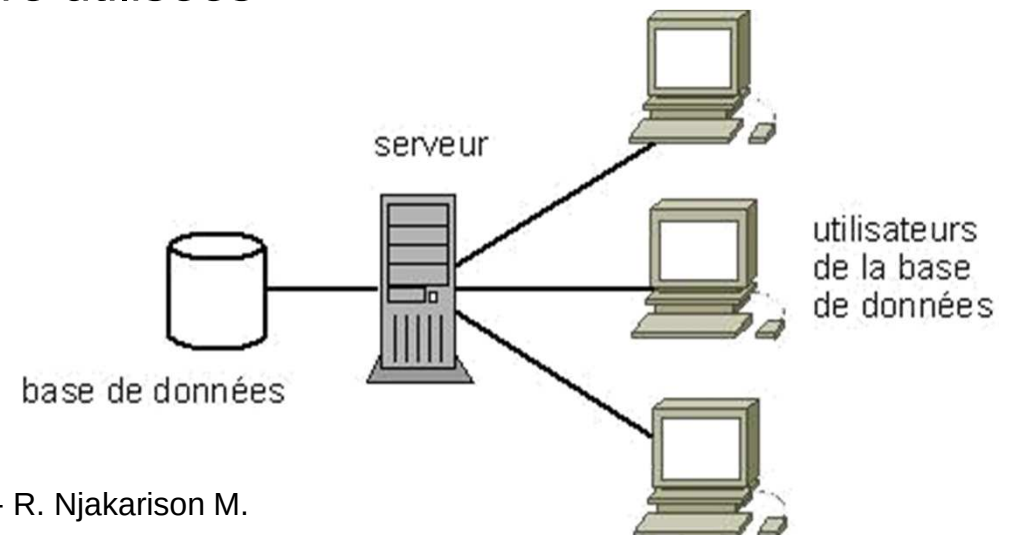
Partie1 Introduction aux Base de Données

1.1. Définitions

a. Base de données (BD)

- Une base de données est un ensemble structuré de connaissance organisée à l'aide d'un formalisme . Elle représente les informations nécessaires au fonctionnement d'une entreprise.
- La gestion des informations est assurée par un logiciel appelé système de gestion de base de données (SGBD)
- Ces données doivent pouvoir être utilisées par des utilisateurs différents.

Ainsi, la base de données est généralement couplée à celle de réseau, afin de pouvoir mettre en commun ces informations.





b. Banque de données

C'est un ensemble de données propre à un domaine d'application que des producteurs réunissent pour ensuite commercialiser l'usage vers le public.

Ex : - Banque de données juridiques
 - Banque de données économiques

La consultation et l'exploitation des banques de données font appel à des techniques spécifiques.

c. Fichier

C'est une unité logique de sauvegarde de données, il y a des cas où chaque type de données correspond à un fichier.

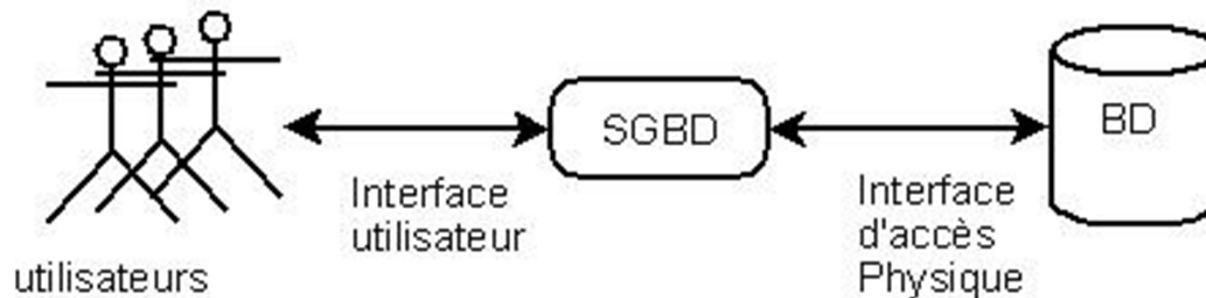
Le logiciel de traitement s'appelle système de gestion de fichier (SGF).

d. Système de gestion de base de données (SGBD)

Le SGBD est un ensemble de services (applications logicielles) permettant de gérer les bases de données, c'est-à-dire :

- permettre l'accès aux données de façon simple
- autoriser un accès aux informations à de multiples utilisateurs
- manipuler les données présentes dans la base de données (insertion, suppression, modification)

Architecture des SGBD



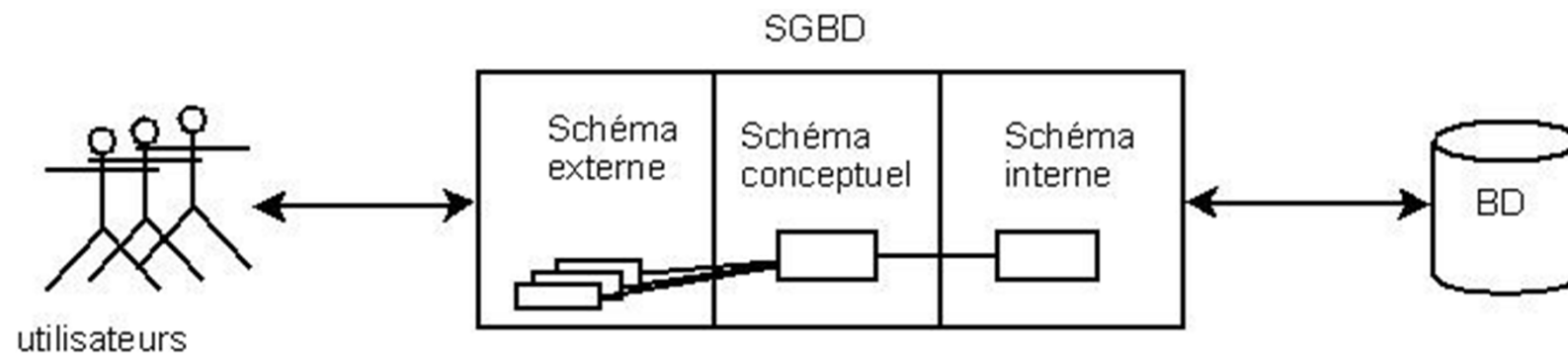
- Interface utilisateur

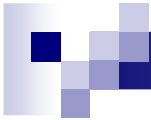
L'interface utilisateur permet

- d'exprimer des requêtes (ou langage de manipulation de données LMD)
- De définir le contenu de la base de données

- Interface d'accès physique

L'interface d'accès physique permet au SGBD d'accéder aux données sur le support physique (Disque,...)





1.2. Les utilité d'une Base de Données

On a besoin d'une base de données lorsque :

- ☐ la quantité de données est importante
- ☐ on a besoin de fiabilité
- ☐ on a besoin d'accéder aux données de façon concurrente
- ☐ on cherche à pérenniser (rendre durable) les résultats de sa recherche
- ☐ on recherche l'indépendance vis-à-vis d'un langage ou d'un programme
- ☐ on a besoin d'interopérabilité



1.3. Les quatre rôles d'une base de données

Une base de données assure

a. La description des données

Décrire les données de l'application sans faire référence à une solution informatique particulière

=>Modélisation conceptuelle

Élaborer une description équivalente pour le stockage des données

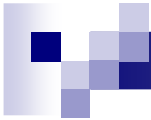
⇒ Modélisation logique

⇒ Langage de description de données

b. L'interrogation des données

Répondre à toute demande d'information portant sur les données contenues dans la base

⇒ Langage de requête (langage d'interrogation)



c. L'exactitude et cohérence (organisation logique des éléments)

Exprimer des contraintes sur les données

⇒ Langage d'expression de contraintes d'intégrité

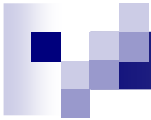
d. Fiabilité et gestion de la concurrence

Pas de perte d'information due à un dysfonctionnement matériel quelconque (erreur de programmation, panne système, panne de l'ordinateur, coupure de courant, ...)

⇒ Garantie de fiabilité

Pas de perte d'information due à l'action d'un utilisateur

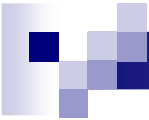
□ Garantie de contrôle de concurrence



1.4. Modélisation d'une base de données

On a besoin de modéliser une base de données

- Pour définir le contenu en informations (connaissances) de la future base de données
- Le modèle en tant que tel est un résultat scientifique car il décrit et résume le « savoir » disponible sur un sujet à un instant donné
- Pour disposer d'un outil de communication accessible aux futurs utilisateurs de la base de données



Partie2 Le processus de modélisation

- Schéma conceptuel (modélisation): contenu de la base en termes de concepts (modèle Entité Association).
 - Schéma logique: « traduction » du schéma conceptuel dans le paradigme du système de base de données (modèle relationnel)
 - Organisation physique: organisation et accès aux données dans la base (modèle SGBD)
- } Spécifications
- } Implémentation



La modélisation conceptuelle

La modélisation conceptuelle est l'étape la plus importante dans l'élaboration d'une base de données dans la mesure où on y détermine le contenu en informations de la future base. Toutes les erreurs faites à ce niveau sont difficiles à corriger.

L'objectif de ce schéma conceptuel est de décrire les objets principaux de la base, leurs caractéristiques et les relations qui les unissent.

Il en existe de nombreuses formalismes, mais nous utiliserons le formalisme entité relation ER (encore appelé entité - association) en raison sa simplicité et de l'adhésion qu'il remporte.

Le modèle ER est simple, il ne comprend que deux concepts : les entités et les relations.



2.1. Modèle conceptuelle de données (MCD)

a. Classification et représentation

Informations

C'est tout élément qui permet de renseigner sur un objet, un concept, un évènement,...

L'information peut prendre plusieurs formes

- Écrite : fiches, documents, fichiers,...
- Symbolique : couleur, taille, forme,...
- Orale

Pour automatiser un système d'information (SI), l'information doit être:

- bien définie et bien représentées
- Classifiée précisément



a. Classification et représentation (suite)

□ Les classifications possibles

Il est possible d'avoir des données :

- Élémentaire $\Rightarrow E$
- Calculée (résultat d'un calcul) $\Rightarrow Ca$
- Concaténée (résultat d'un concaténation) $\Rightarrow Co$

□ La Représentations

Les informations peuvent être représentées en :

- Alphabétiques (des textes entiers) $\Rightarrow A$
- Alphanumériques (chaîne de caractère numérique : dates,...) $\Rightarrow AN$
- Numériques (des numérotations, chiffres, ...) $\Rightarrow N$
- Booléens (Oui ou Non) $\Rightarrow B$



b. Dictionnaire de données (DD)

- Le dictionnaire de données contient toutes les informations représentables pour concevoir une BD.
- Les informations dans un DD ne doivent pas être représentées par des synonymes ni des mots qui possèdent plusieurs significations.
- On représente un DD dans un tableau

<i>Propriété</i>	<i>Contenu</i>	<i>Nature</i>	<i>Type</i>
Désignation à donner à l'information <i>Ex :</i> Nom_etu	Description de la propriété Nom d'un étudiant	Représentation de l'information (A, AN, N, B) A	Classification à donner à l'information (E, Ca, Co) E



b. Dictionnaire de données (suite)

Exemple :

Une coopérative de transport à DS veut informatiser la gestion des circulations de ses véhicules. Cette coopérative a besoin stocker les informations sur :

- Le voyage que doit effectuer chaque véhicule (numéro de voyage, date , heure, lieu de départ, destination, nombre de passagers à bord);
- Les véhicules en service (numéro, modèle, marque, vitesse, capacité)
- Les conducteurs (numéro du conducteur, nom, sexe,)

Question :

Établir un DD correspondant à ce problème

Exemple:

<i>Propriété</i>	<i>Contenu</i>	<i>Nature</i>	<i>Type</i>
<i>Num_voy</i>	Numéro de voyage	N	E
<i>Date_voy</i>	Date du voyage	AN	E



DD

<i>Propriété</i>	<i>Contenu</i>	<i>Nature</i>	<i>Type</i>
<i>Num_voy</i>	Numéro de voyage	N	E
<i>Date_voy</i>	Date du voyage	AN	E
<i>Heure_dep</i>	Heure de départ du voyage	AN	E
<i>Lieu_dep</i>	Lieu de départ	A	E
<i>Dest_voy</i>	Destination	A	E
<i>Nbr_pass</i>	Nombre de passagers	N	E
<i>Num_veh</i>	Numéro du véhicule	AN	E
<i>Mod_veh</i>	Modèle du véhicule	A	E
<i>Marque_veh</i>	Marque du véhicule	A	E
...			
<i>Id_cond</i>	Identifiant du conducteur	N	E
<i>Nom_cond</i>	Nom du conducteur	A	E
<i>Sex_cond</i>	Sexe du conducteur	A	E
...			



c. Dépendances fonctionnelles (DF)

Pour établir efficacement un modèle entité association, on doit étudier au préalable les dépendances fonctionnelles entre les attributs

Un attribut peut contenir un ensemble de valeur (ou d'informations) indépendants

□ **Définition**

*Un attribut **Y** dépend fonctionnellement d'un attribut **X** si et seulement si une valeur de **X** induit une unique valeur de **Y**.*

On note une dépendance fonctionnelle par : $X \rightarrow Y$

Exemple:

⇒ Si X est le Id_cond (identifiant du conducteur) et Y le nom_cond (nom du conducteur) alors on a : $X \rightarrow Y$ car si on connaît l'Id du conducteur on peut connaître son nom. Par contre, on a pas $Y \rightarrow X$, car plusieurs conducteur peuvent porter le même nom.

⇒ Mod_veh ne dépend pas fonctionnellement du Marque_veh car la connaissance du modèle de véhicule ne donne pas sa marque

- **Transitivité d'une DF:**

Si $X \rightarrow Y$ et $Y \rightarrow Z$ alors $X \rightarrow Z$

- **DF directe**

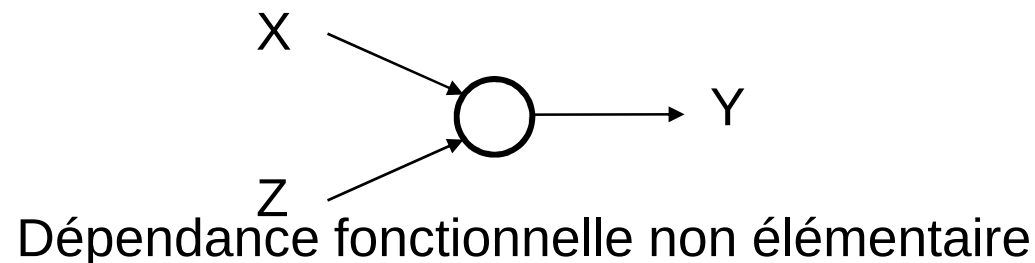
Une DF est dite directe s'il n'y a pas de transitivité entre les attributs qui dépendent fonctionnellement

Si $X \rightarrow Y$ et $Y \rightarrow Z$ alors $X \rightarrow Z$

$X \rightarrow Y$ et $Y \rightarrow Z$ sont des DF directes par contre $X \rightarrow Z$ n'est pas directe.

- **DF élémentaire et non élémentaire**

Un attribut Y peut avoir une dépendance fonctionnelle qui repose sur la conjonction de plusieurs attributs, dans ce cas la dépendance est dite **non élémentaire**. Si Y repose sur un seul attribut X alors la dépendance est élémentaire

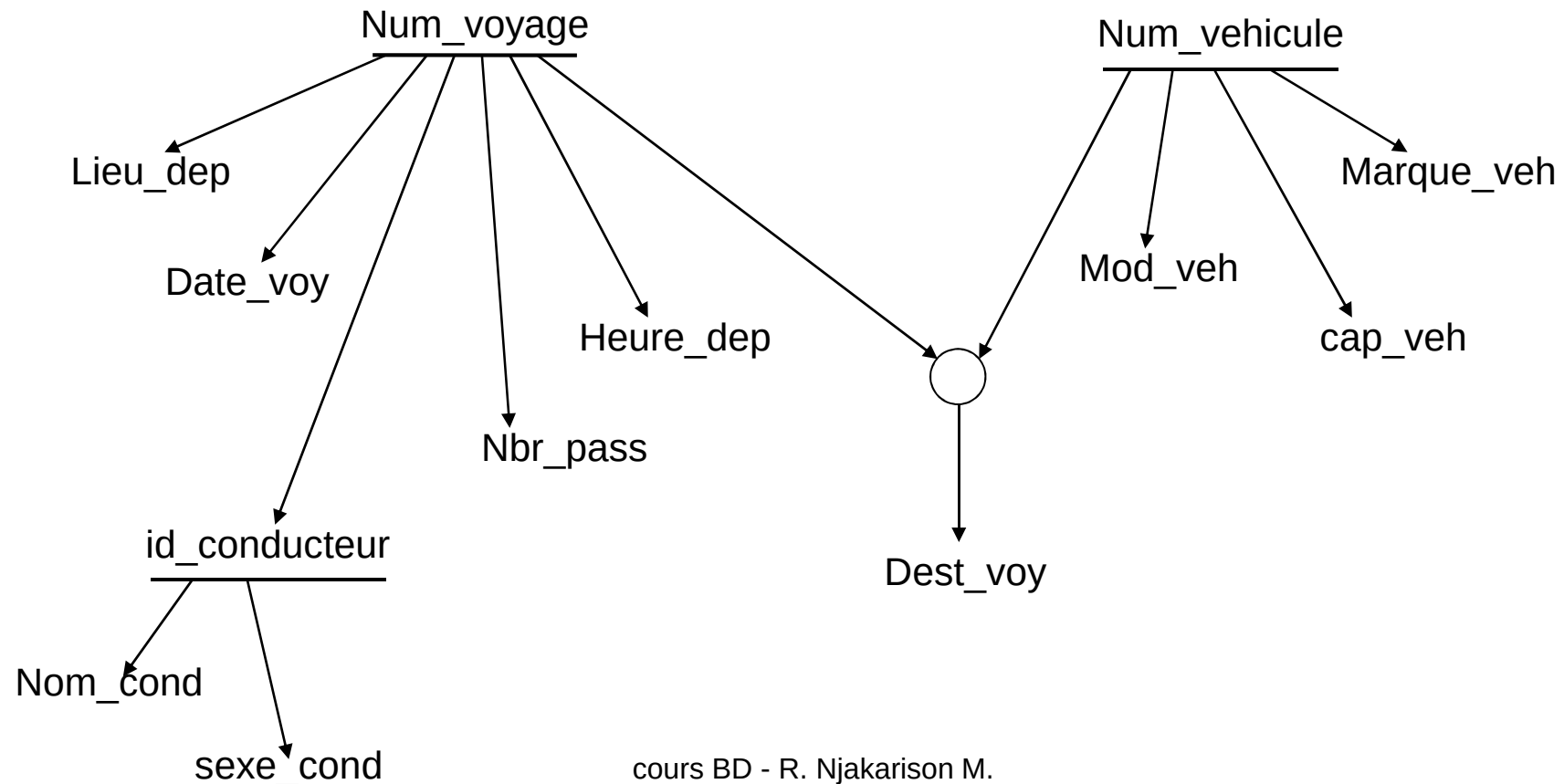


Aussi appelée DF à partie gauche composé (DFPGC)

d. Graphe de couverture minimale ou

Graphe de dépendances fonctionnelles (GCM ou GDF)

Une GCM ou GDF est un graphe qui permet de représenter le diagramme de tous les DF simple, élémentaire, directe et à partie gauche composé (on souligne toutes les sources de la DF).





e. Le Modèle Entité Association

Entité

Une entité est une population d'individu homogène. Par exemple les produits ou les articles vendus par une entreprise peuvent être regroupés dans une même entité **article**, car d'un article à l'autre les informations ne changent pas de nature.

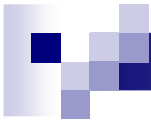
Une entité est représentée par un tableau à 2 lignes et 1 colonne et dans la 1^{re} ligne on met le nom de l'entité

Exemples :

Voyage

Véhicule

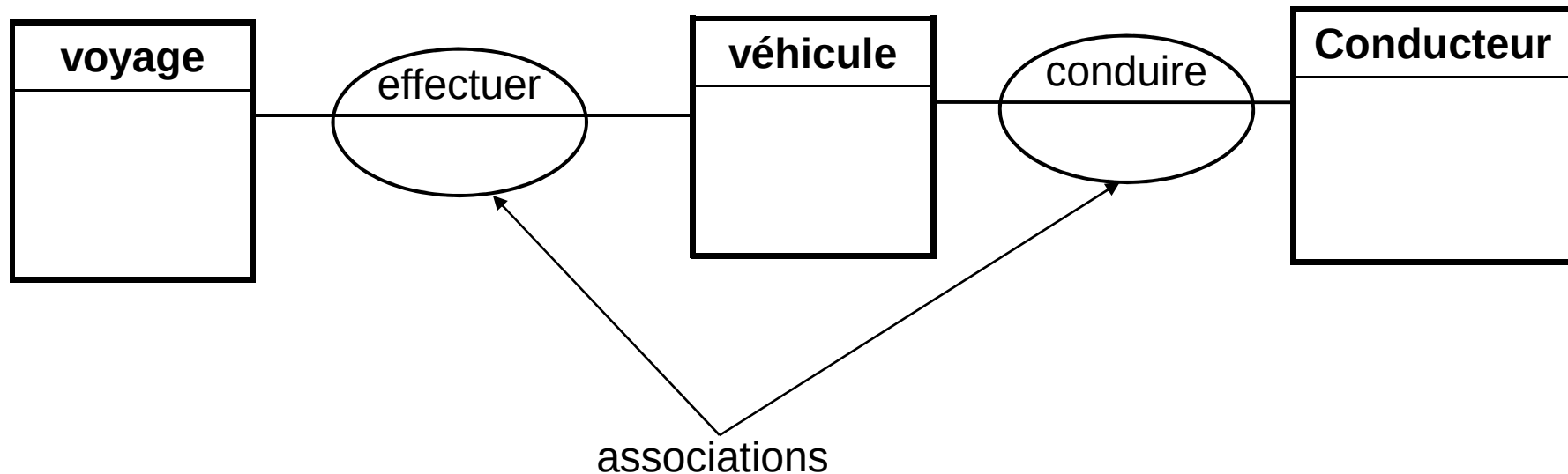
Conducteur



- Association

Une association est une liaison qui a une signification précise entre plusieurs entités :

Alors la liaison entre les entités ne sont pas directes



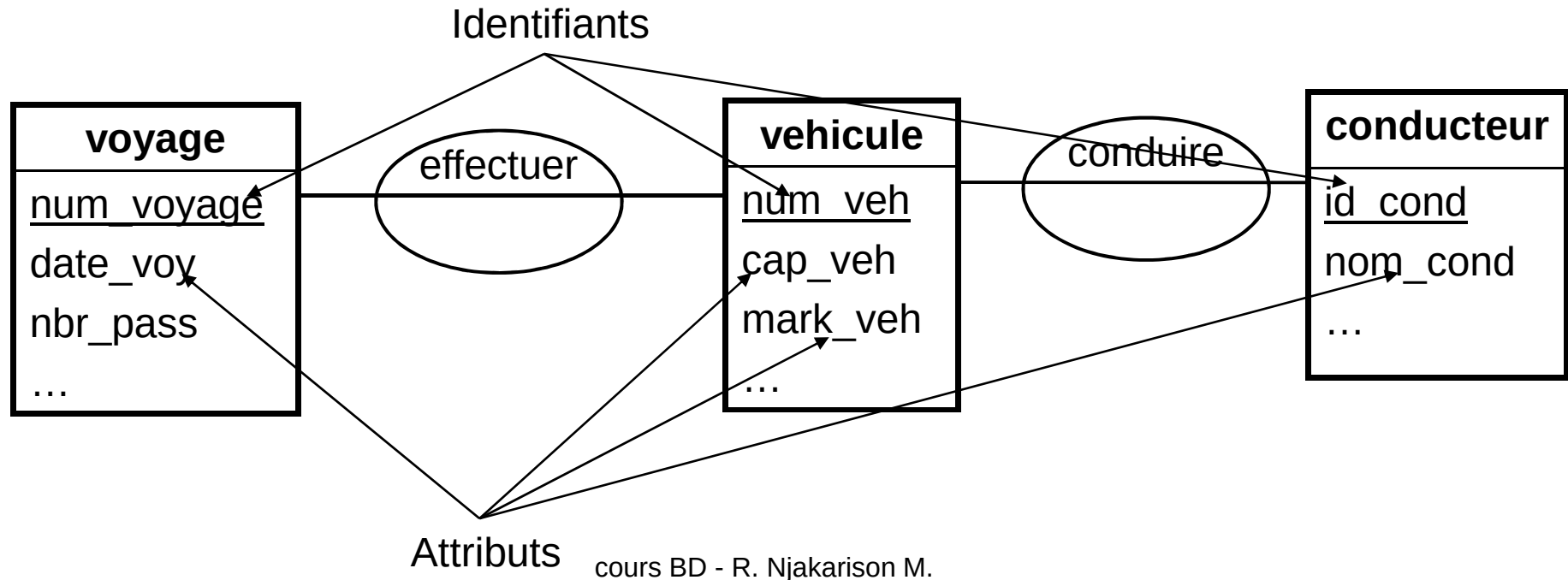
- Attribut et identifiant

Un attribut est une propriété d'une entité ou d'une association

Un identifiant est un attribut sans doublon. Par convention, on souligne l'identifiant dans un schéma ER.

NB

- Une entité possède au moins un attribut (son identifiant)
- Au contraire une association peut être dépourvue d'attribut



- Cardinalité

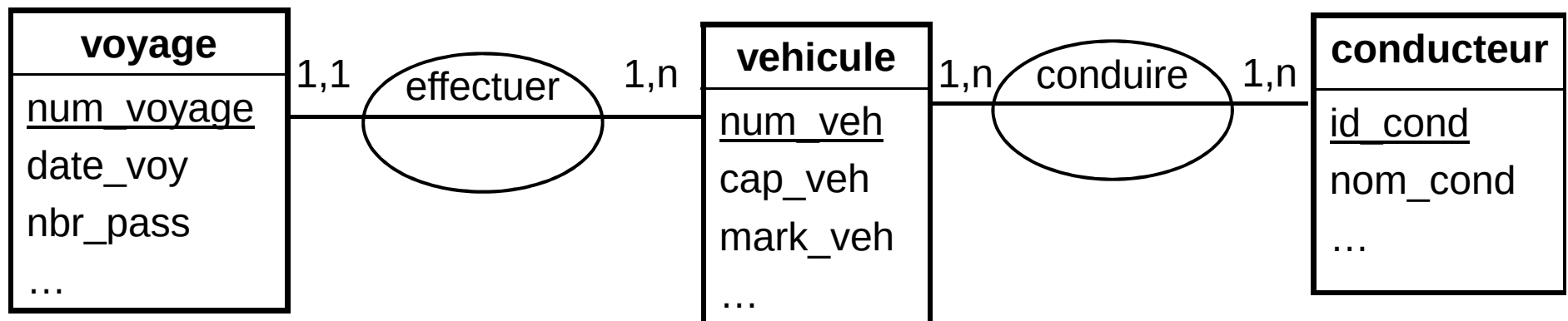
La cardinalité d'un lien entre une entité et une association précise le minimum et le maximum de fois qu'un individu de l'entité peut être concerné par l'association. Pour établir une cardinalité correctement, il faut poser les questions dans le bon sens

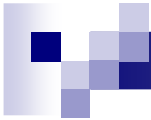
ex : entre voyage et véhicule

Question Coté voyage : une voyage peut être effectué par combien de véhicule? **La Réponse** : 1 et 1 seul (1,1)

Question coté véhicule : un véhicule peut effectuer combien de voyage.

La réponse : 1 ou plusieurs (1,n)





f. Traduction d'un GDF en un schéma ER

À partir du GDF, le schéma ER normalisé correspondant apparaît en suivant quelques étapes simples.

Étape 1: les attributs source de dépendance (souligné) sont des identifiants

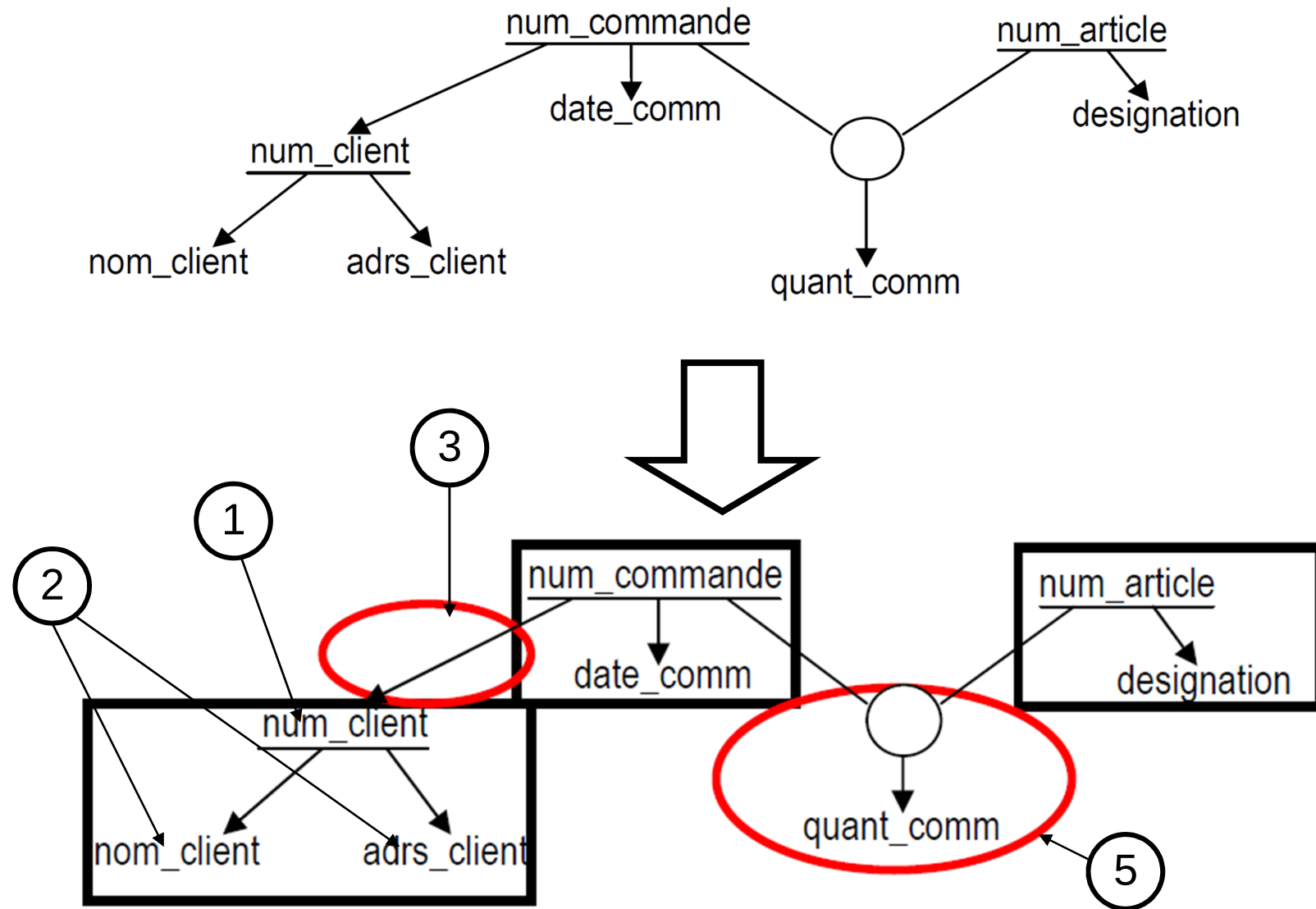
Étape 2: les attributs non identifiants qui dépendent directement d'un identifiant forment une entité avec l'identifiant source

Étape 3: les dépendances élémentaires entre les identifiants forment des associations binaires dont les cardinalités maximales sont 1 au départ de la dépendance fonctionnelle et n à l'arrivée.

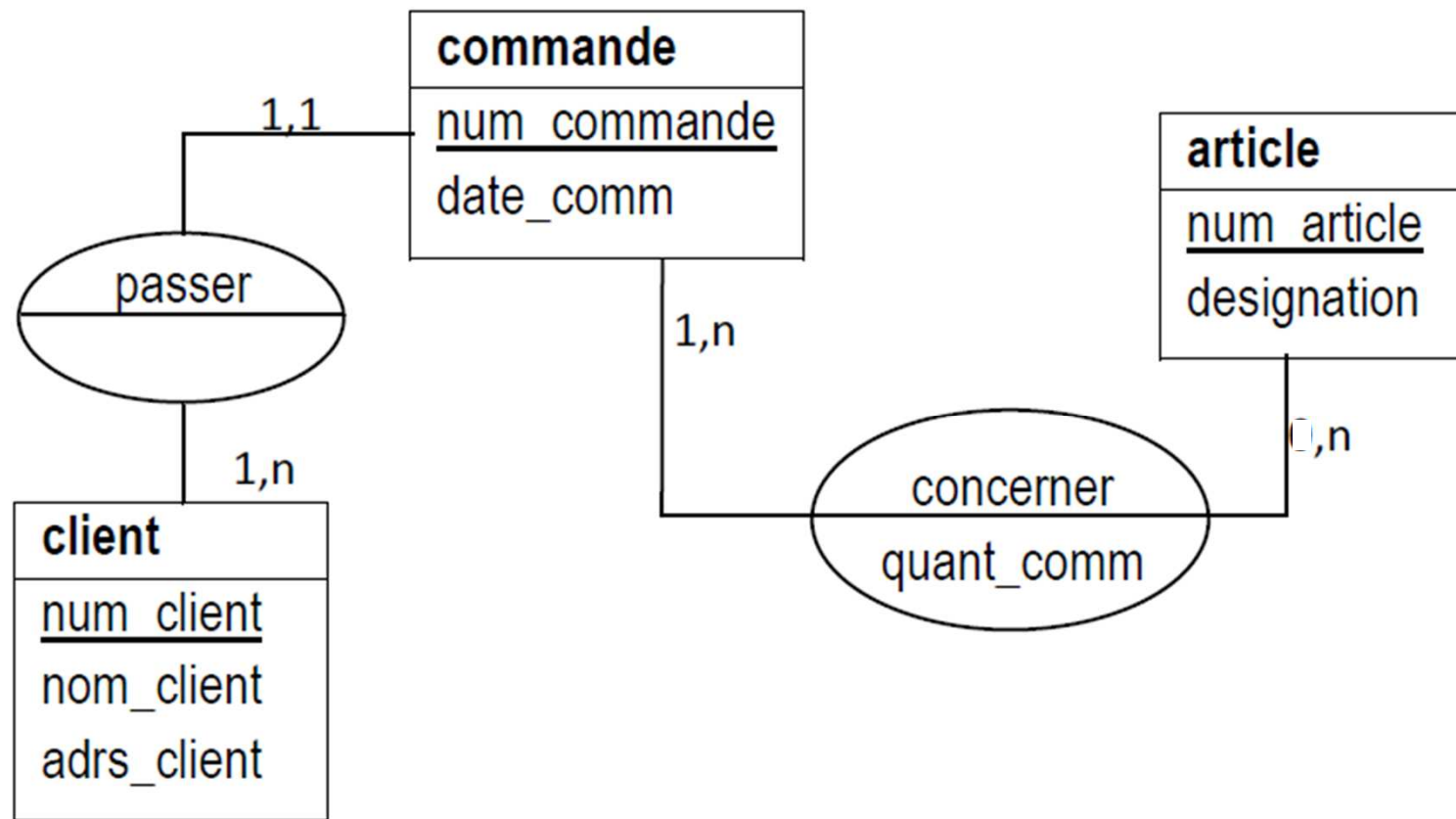
Étape 4: si entre deux identifiants, il y a deux dépendances élémentaires réflexives, dans ce cas l'association binaire à 2 cardinalité maximale 1.

Étape 5: les attributs (non identifiants) qui dépendent de plusieurs identifiants sont les attributs d'une association supplémentaire dont les cardinalités maximales sont toutes n

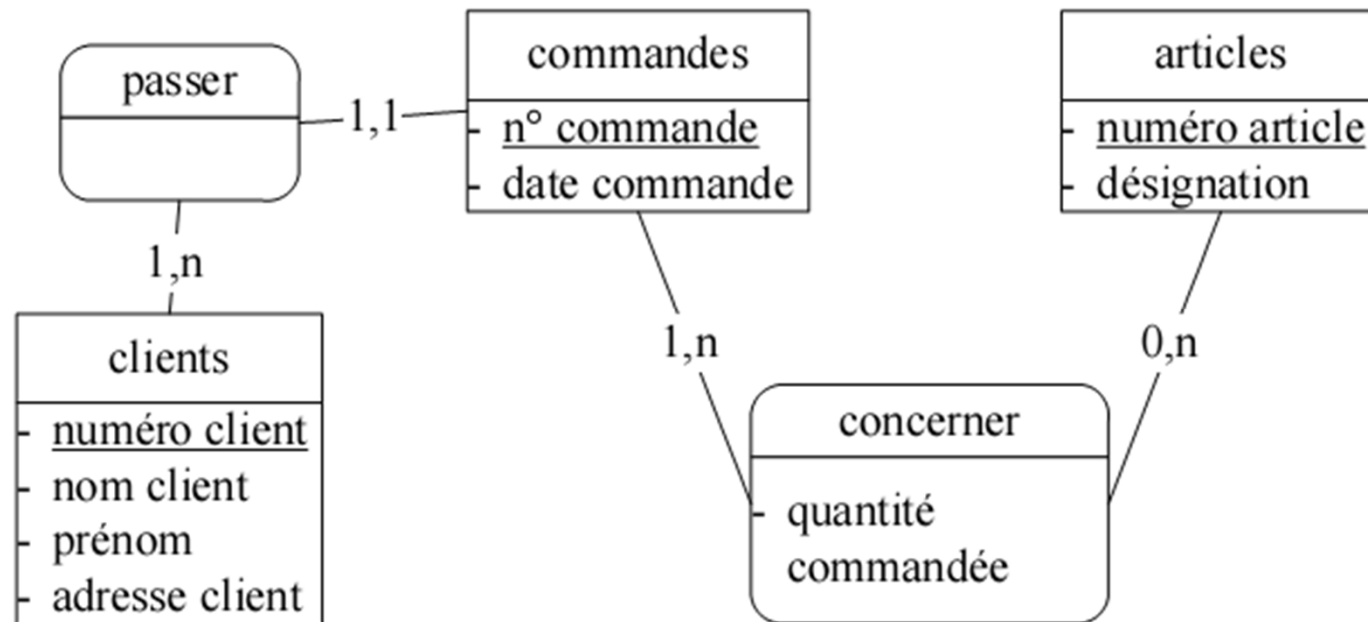
Exemple de Traduction d'un GDF en un schéma EA



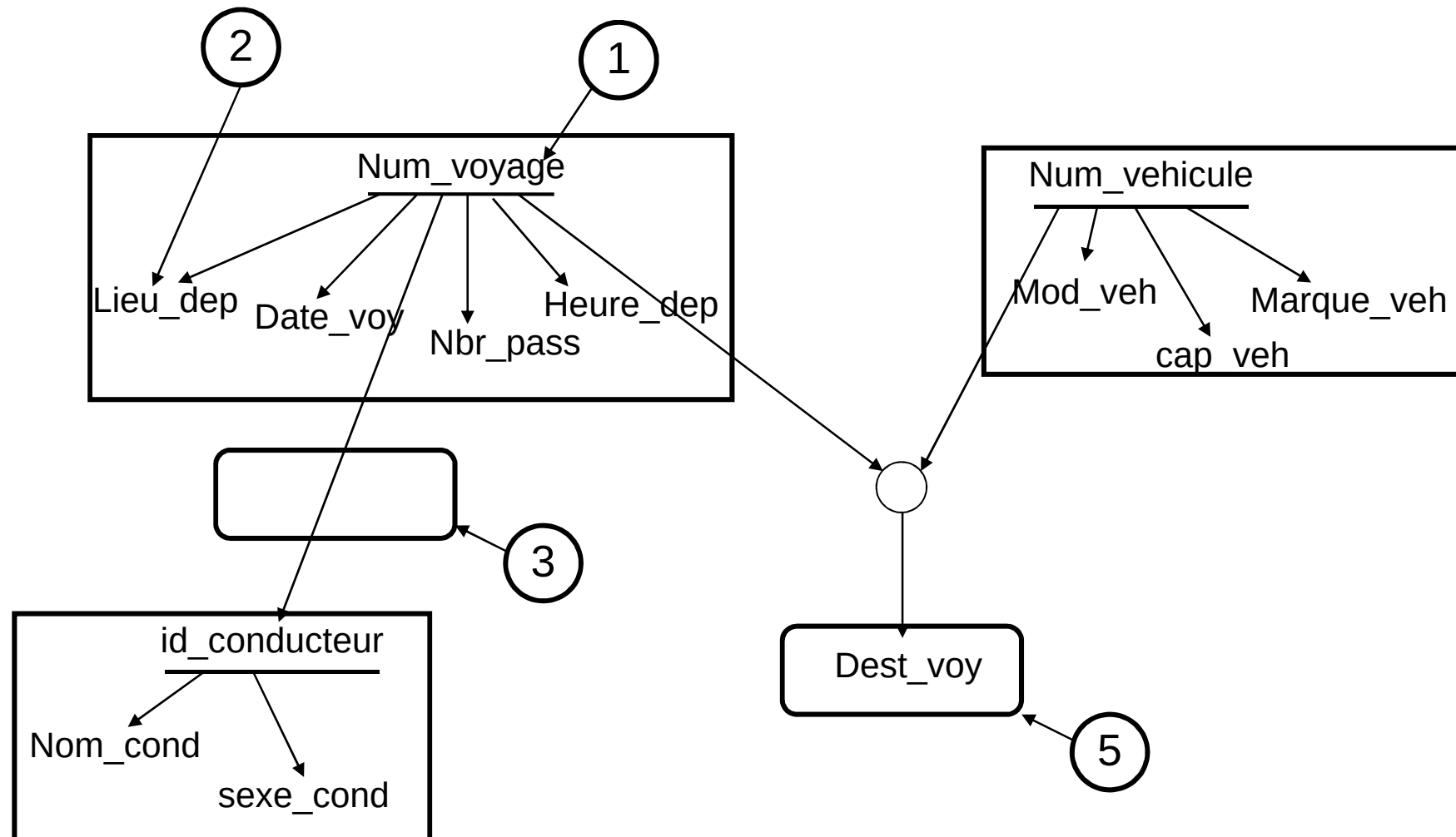
Finalement on a le schéma EA suivant



Finalement on a le schéma ER suivant



Traduction du GDF en schéma ER

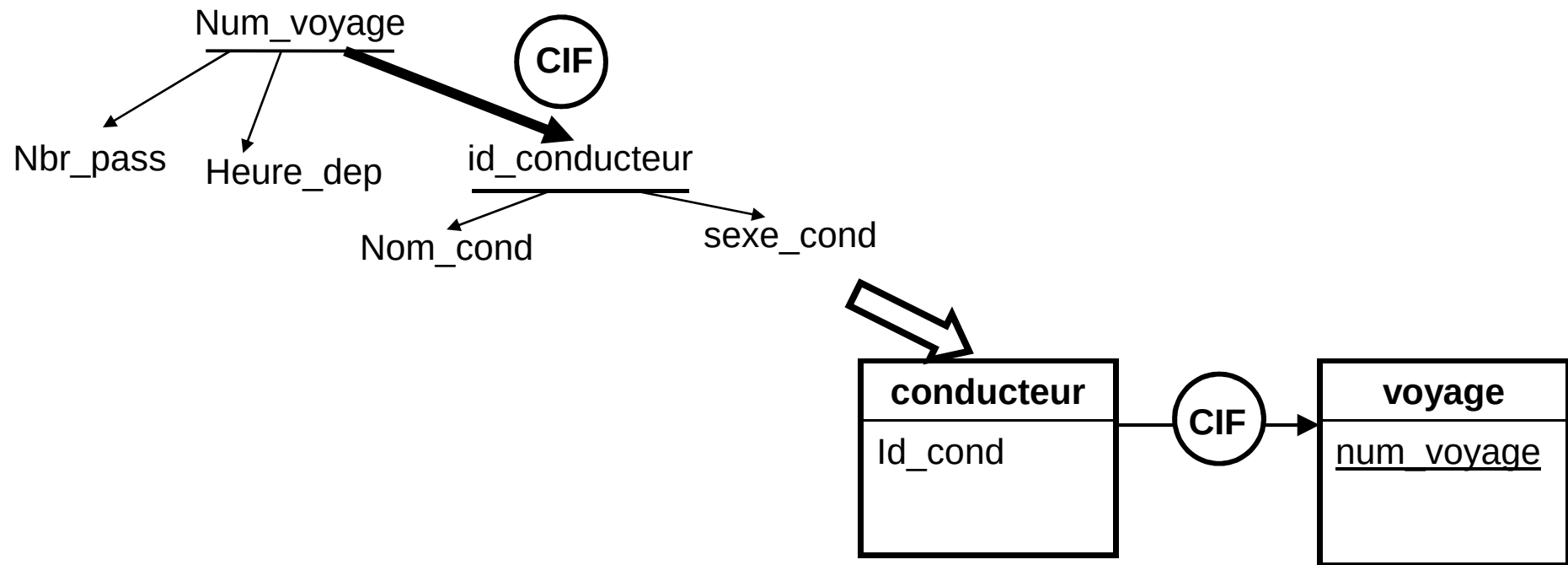


- Contrainte d'Intégrité Fonctionnelle (CIF)

CIF binaire

À l'issue de l'étape 3, sur la transformation d'un GDF en schéma ER, une CIF est formé si une source de DF (non DFPGC) pointe vers un attribut source de DF ou DFPGC

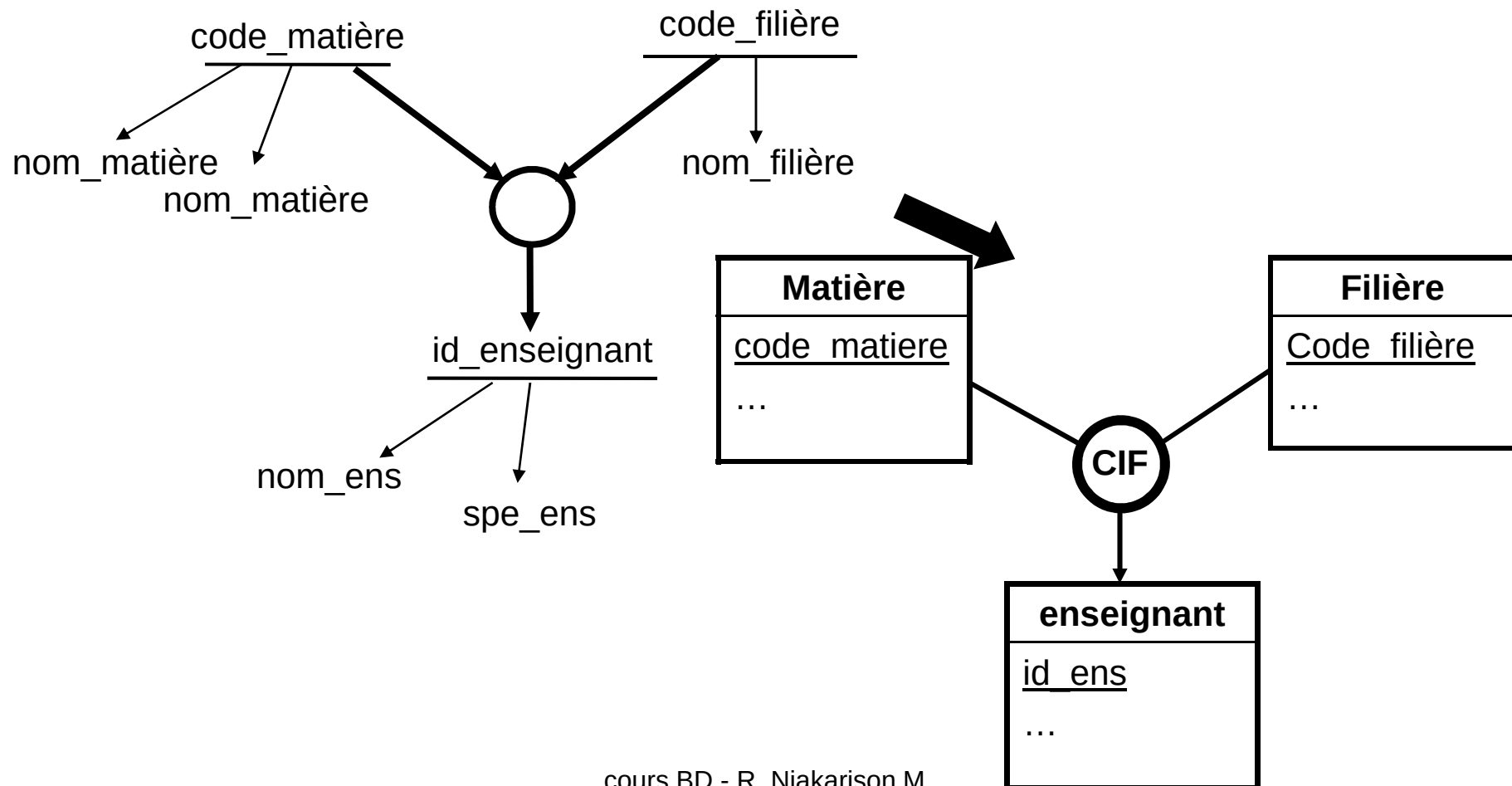
Exemple:



CIF multiple:

Une DFPGC pointant vers un attribut source de DF implique une CIF multiple

exemple





Résumé des Règles de passage d'un GDF ➡ MCD

- création du graphe
- détermination des sources de dépendances
- Détermination des entités
- détermination des CIF binaires
- formation des autres associations
- déterminer les CIF multiples
- ajouter les attributs qui sont nécessaire dans le MCD



g. Modèle Relationnel (MR)

Concepts d'un MR :

Le modèle relationnel possède 3 concepts essentiels

- ***Le relation***
- ***Les attributs***
- ***Le domaine***

Un Domaine est un ensemble dénombrable de valeurs d'un attribut d'une relation

Le produit cartésienne d'un domaine D1 avec un domaine D2 forme l'ensemble de tous les couples $\langle v1, v2 \rangle$ où $v1 \in D1$ et $v2 \in D2$.

Exemple :

Couleur_vin (rouge, blanc, rose)

Nom_vin(lazan'i Betsileo, côte de Fianara, soavita)

Couleur_vin x Nom_vin($\langle$ rouge, lazan'i Betsileo $\rangle$, $\langle$ blanc, lazan'i Betsileo $\rangle$, $\langle$ rose, lazan'i Betsileo $\rangle$, $\langle$ rouge, côte de Fianara $\rangle$, $\langle$ blanc, côte de Fianara $\rangle$, ...)

- Relation

Une relation est un sous ensemble du produit cartésienne d'une liste de domaines. Elle est représenté par une table à deux dimensions et chaque ligne de la table est appelée ***tuple*** et chaque colonne correspond à un domaine.

Nom de la relation :
Hôtel

Attributs ou Colonnes

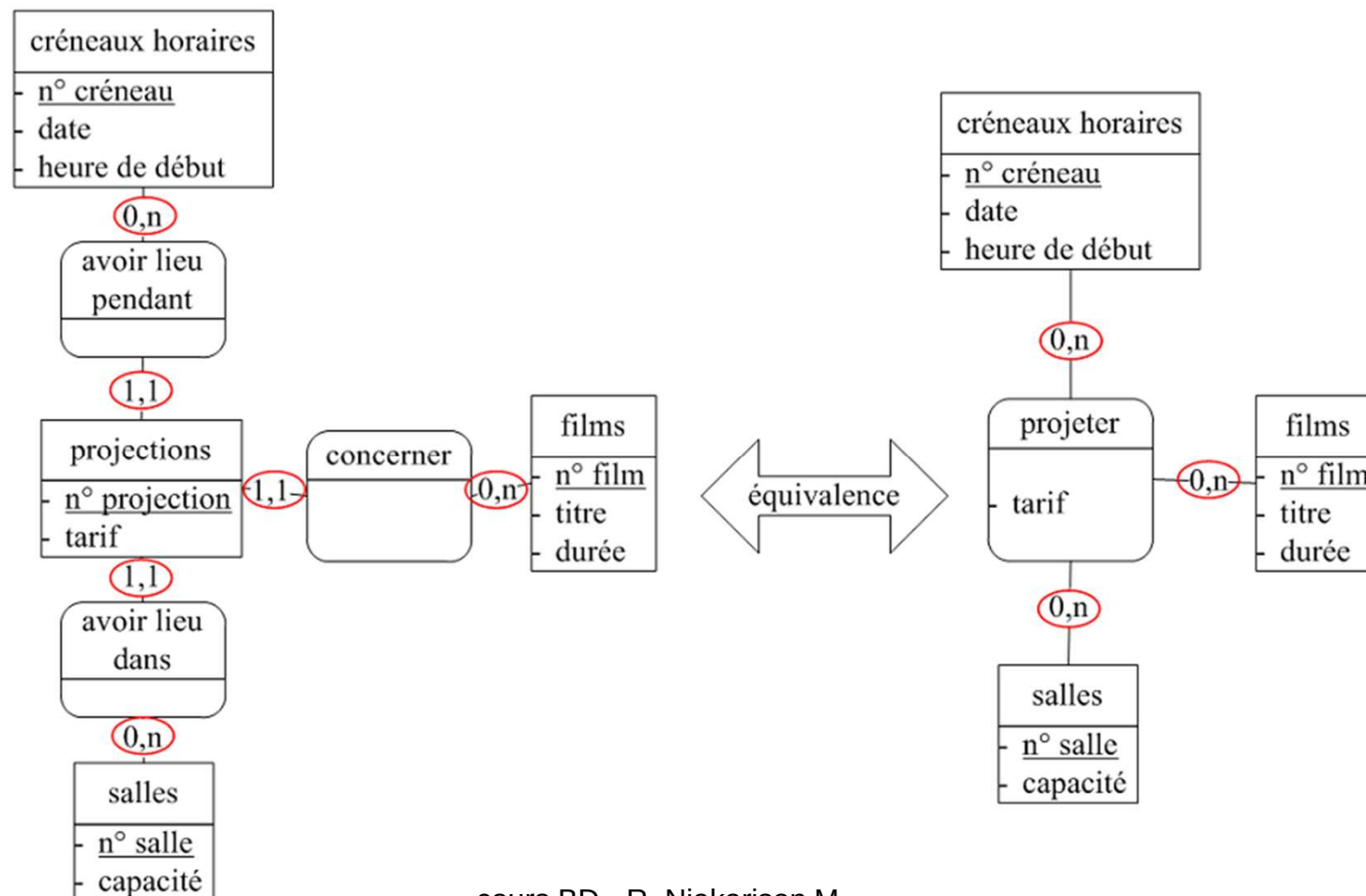
<u><i>Id_hotel</i></u>	<i>Nom_hotel</i>	<i>localisation</i>	<i>Class_hotel</i>	<i>Dist_ville</i>	
01	Grand Hôtel	Rue Colbert	4 étoiles	0,8 Km	
05	La Note Bleu	Route Ramena	3 étoiles	5 Km	<i>Tuple</i>
06	Hôtel de la poste	Place Kabary	3 étoiles	1 Km	
11	Emeraude	Lazaret	3 étoiles	0,9 Km	
		<i>Domaine</i>			

Exemple de Schéma relationnel

h. Les règles normalisations

Un schéma entités - associations doit répondre 9 règles de normalisation.

R1. Normalisation des entités : toutes entités remplaçable par des associations doivent être remplacés





R2. Normalisation des noms : le nom d'une entité, d'une association ou d'un attribut doit être unique.

- Entités : utiliser un nom commun (au pluriel) ex : clients, personnes
- Association : utiliser un verbe à l'infinitif (ex : effectuer, concerner,...) à la forme passive (être commandé, ...) et accompagné d'un adverbe (avoir lieu dans, pendant, à, ...).
- Attributs : utiliser un nom commun singulier ex: nom, numéro
...accompagné (ou concaténé) du nom de l'entité dans laquelle il se trouve.

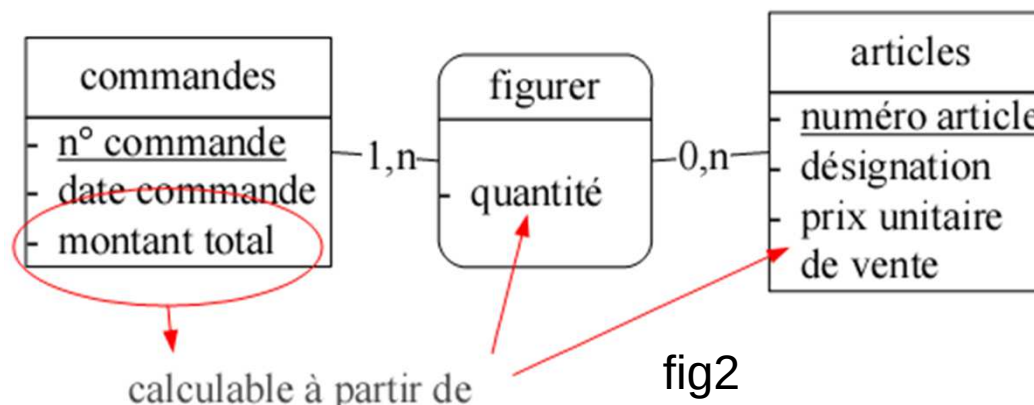
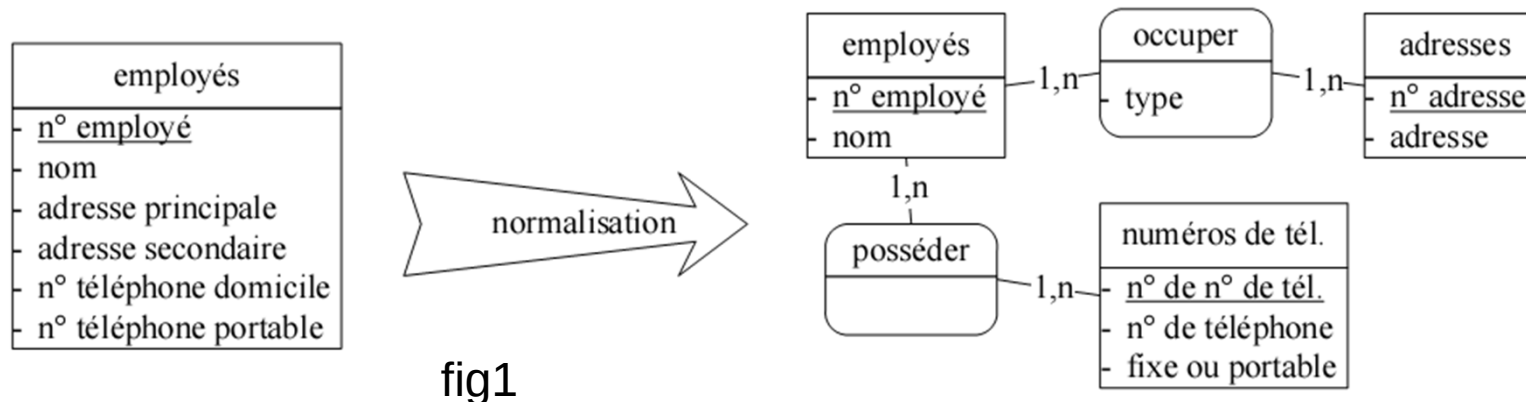
R3. Normalisation des identifiants

Chaque entité doit posséder un identifiant

- Éviter un identifiant composé de plusieurs attributs
- Utiliser un identifiant court pour rendre la recherche plus rapide
- Éviter les identifiant qui peuvent changer au cours du temps

R4. a. Normalisation des attributs :

Remplacer les attributs en plusieurs exemplaires en une association supplémentaire de cardinalité maximale n (fig1) ; ne pas ajouter d'attribut calculable à partir d'autre attributs (fig2)



R4. b. Normalisation des attributs d'une association : les attributs d'une association doivent dépendre directement des identifiants de toutes les entités en associations

R5. Normalisation des associations : il faut éliminer les associations fantômes (fig1), redondantes ou en plusieurs exemplaires (fig2)

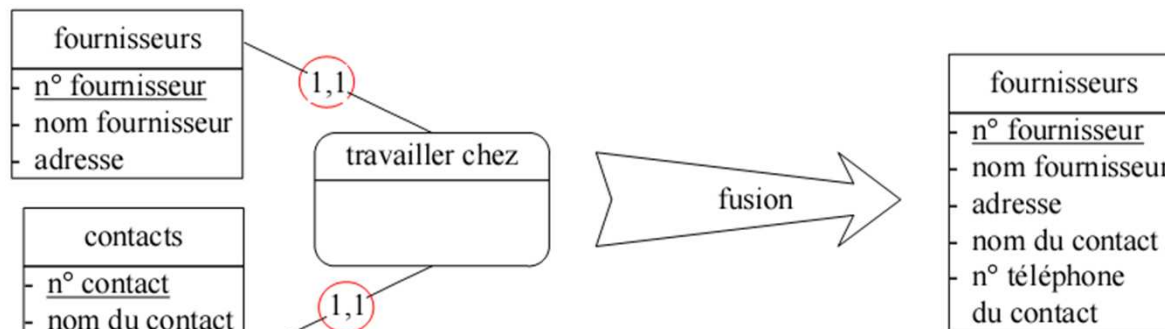


fig1

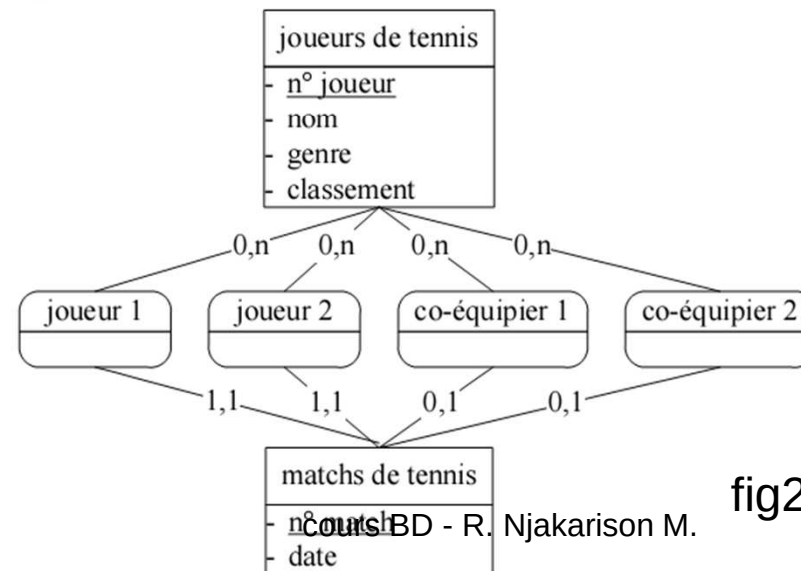


fig2



R6. Normalisation des cardinalités : une cardinalité minimale vaut 0 ou 1 (pas 2, 3 ou n) et une cardinalité maximale est toujours 1 ou n (pas 2,3...)

Si une cardinalité maximale est connue (>1) il faut la considérer comme n

On ne peut pas modéliser une cardinalité minimale qui vaut >1

Si une cardinalité maximale vaut 0 alors l'association n'aurait aucune signification.

À ces 6 formes de normalisation, il convient d'ajouter les 3 premières formes normales. Pour qu'une base de données soit optimisée (non redondance ou répétition de données), il faut que les relations soient en 3°

Forme normale de Boyce-Codd

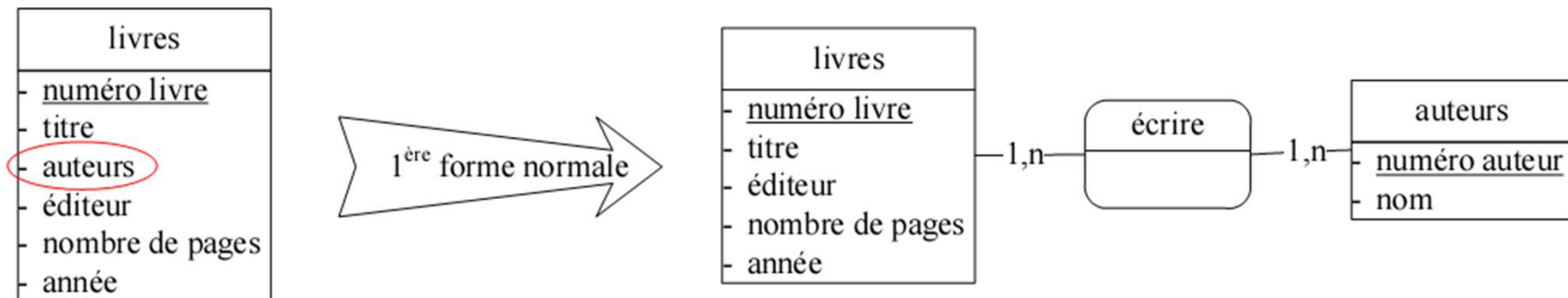
Les formes normales :

R7. Première forme normale (1FN) :

Déf. : Une table est en 1FN si tout les attributs sont en DF avec la clé de cette table.(au niveau du GDF la dépendance doit être fonctionnelle)

Si un attribut de cette table prend plusieurs valeurs alors ces valeurs doivent faire l'objet d'une entité supplémentaire, en association avec la première

Exemple d'application de la 1FN





R8. Deuxième Forme Normale (2FN) :

Déf. :

- Une table est en 2FN si elle est en 1FN
- Son identifiant peut être composé de plusieurs attributs mais les autres attributs de l'entité doivent dépendre de l'identifiant en entier mais non pas une partie de cet identifiant.

Au niveau du GDF, la dépendance doit être fonctionnelle et élémentaire.

R9. Troisième Forme Normale de Boyce-Codd (FNBC ou 3FN) :

Déf. :

- Une table est en 3FN si elle est en 2FN
- Tous les attributs doivent dépendre directement de l'identifiant mais pas et d'aucun autre attribut. Si ce n'est pas le cas il faut placer une entité séparé mais en association avec la première

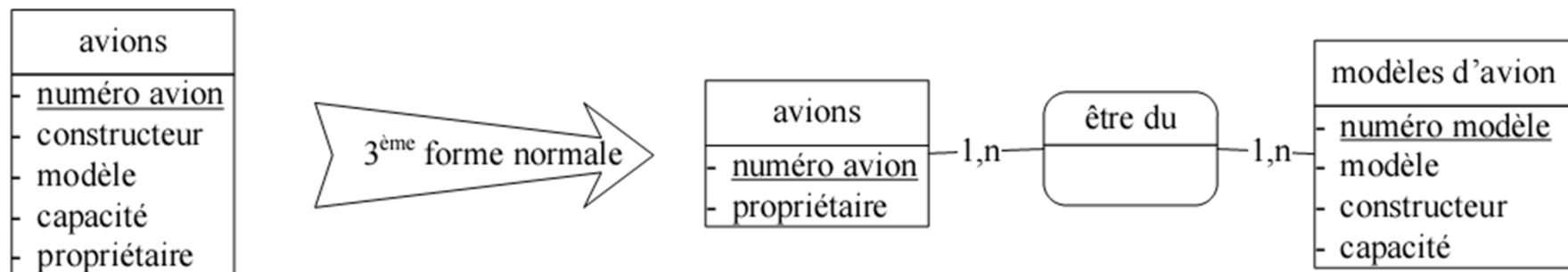
Au niveau du GDF, la dépendance doit être élémentaire et directe.

Application du 3FN

L'entité avion n'est pas en 3FN car la capacité et le constructeur d'un avion ne dépend pas du numéro d'avion mais de son modèle. Il faut alors appliquer la R9 concernant le 3FN

numéro avion	constructeur	modèle	capacité	propriétaire
1	Airbus	A380	180	Air France
2	Boeing	B747	314	British Airways
3	Airbus	A380	180	KLM

application du 3FN





2.2. Modèle Logique de Données MLD

Lorsque les données ont la même structure, on peut les organiser en table dans laquelle les colonnes décrivent les champs en commun et les lignes contiennent les valeurs de ces champs pour chaque enregistrement

Exemple : les renseignements relatifs aux clients

Client(numéro client , nom , prénom , adresse)

numéro client	nom	prénom	adresse
1	Dupont	Michel	127, rue...
2	Durand	Jean	314, boulevard...
3	Dubois	Claire	51, avenue...
4	Dupuis	Marie	2, impasse...
...	...	...	...

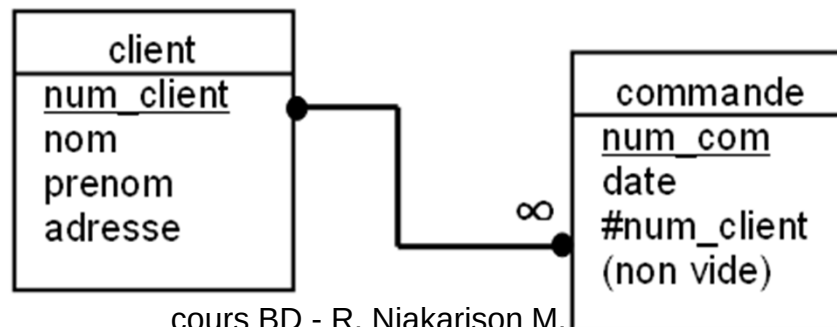
Clés primaires : Les lignes d'une table doivent être uniques. Une colonne (au moins) doit servir à les identifier et c'est la clé primaire de la table.

Clés étrangères : Il se peut qu'une colonne CT1 d'une table T1 ne doive contenir que des valeurs prises par la colonne CT2 d'une autre table T2. CT2 doit être sans doublon ou unique dans T2 (il s'agit souvent d'une clé primaire). On dit alors que **CT1** est **clé étrangère** et qu'elle **référence** la colonne **CT2**.

Par convention on fait précéder les clés étrangères d'un dièse #
représentations :

client(num_client , nom , prénom , adresse)

commande(num_com , date_com , #num_client(non vide))





Remarques :

- Une table peut avoir plusieurs clés étrangères mais une seule clé primaire (qui peut être composé de plusieurs colonnes)
- Une colonne clé étrangère peut aussi être primaire (dans la même table)
- Une clé étrangère peut être composé (si elle est référencé à une clé primaire composée)
- Une colonne clé primaire ne peut recevoir la valeur vide (NOT NULL)
- Si une colonne clé étrangère ne doit pas recevoir la valeur vide, on doit la préciser dans la description des colonnes.



Traduction d'un MCD en Modèle Logique de Données MLD

Pour traduire un MCD en MLD il suffit d'appliquer cinq (5) règles :

Notations : on dit qu'une association binaire (entre deux entités ou réflexive) est de type :

1:1 (un à un) si aucune des deux cardinalités maximales n'est n ;

1:n (un à plusieurs) si une des deux cardinalités maximales est n ;

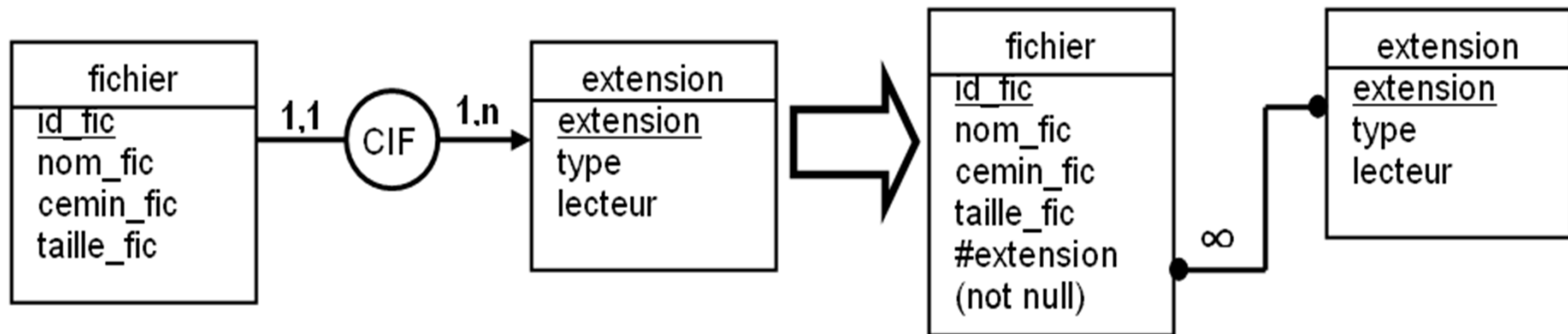
N:m (plusieurs à plusieurs) si les deux cardinalités maximales sont n ;

Règle N°1

Toute entité devient une table dans laquelle les attributs deviennent les colonnes. L'identifiant de l'entité constitue alors la clé primaire de la table.

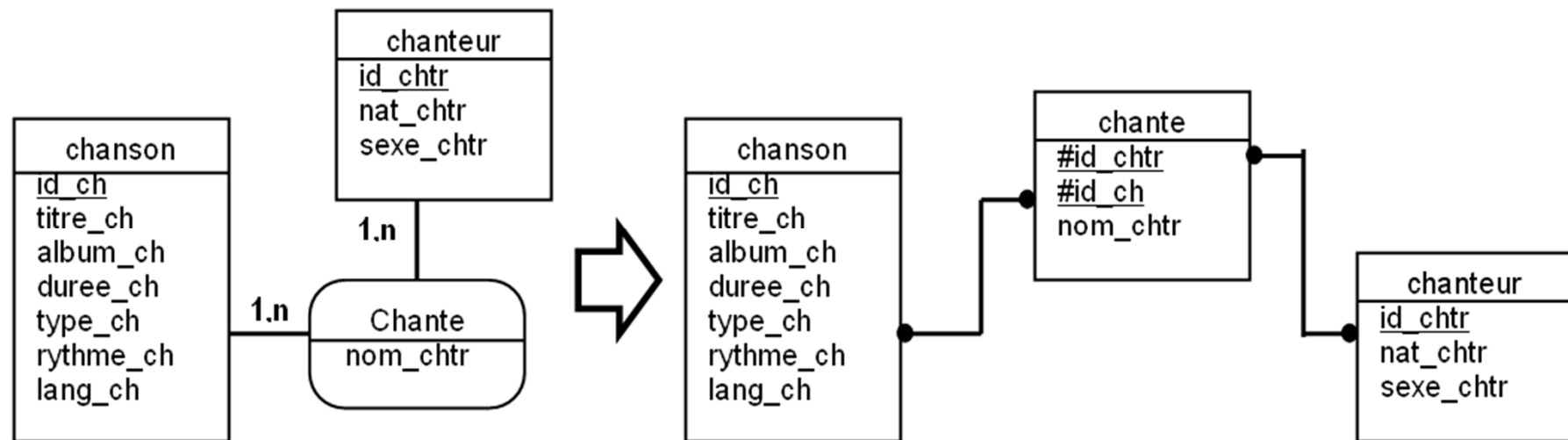
Règle N°2

Une association binaire de type 1:n (CIF binaire) disparaît au profit d'une clé étrangère dans la table coté 0,1 ou 1,1 qui référence la clé de l'autre table. Cette clé étrangère ne peut pas recevoir la valeur vide si la cardinalité est 1,1



Règle N°3

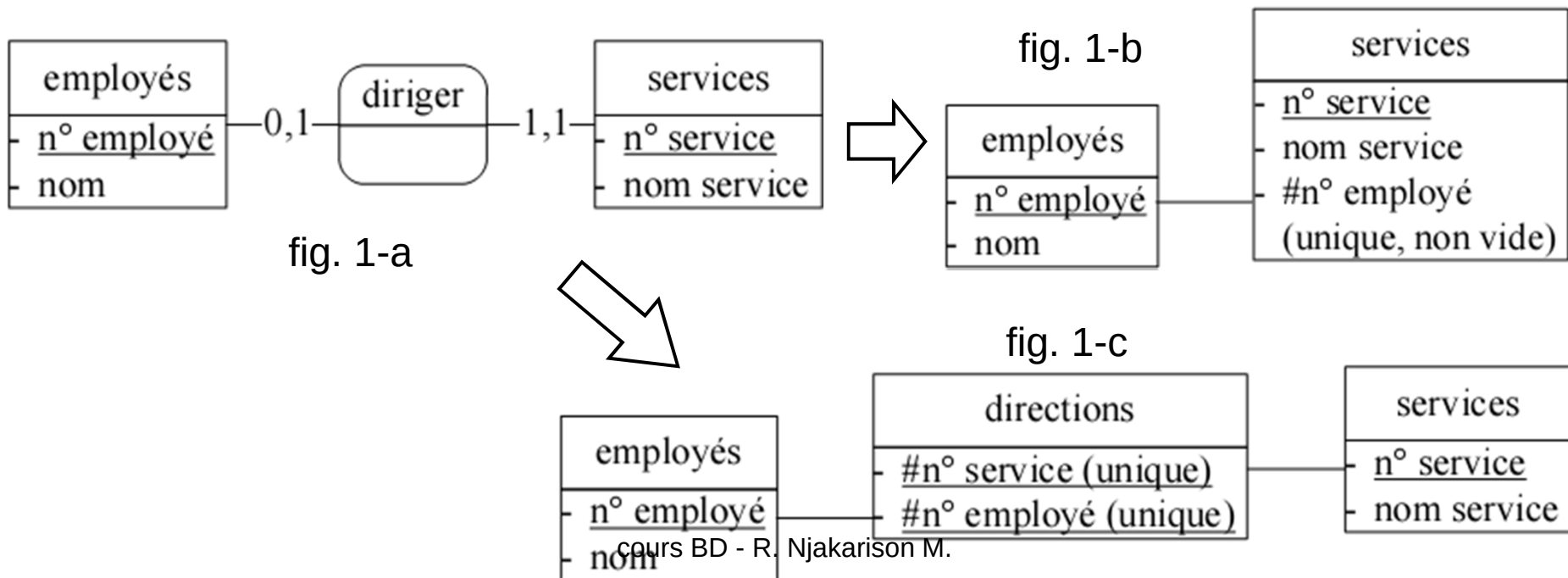
Une association binaire de type n:m devient une table supplémentaire dont la clé primaire est composé de deux clés étrangères (qui référencent les deux clés primaires des deux tables en association). Les attributs de l'association deviennent des colonnes de cette nouvelle table.



Règle N°4

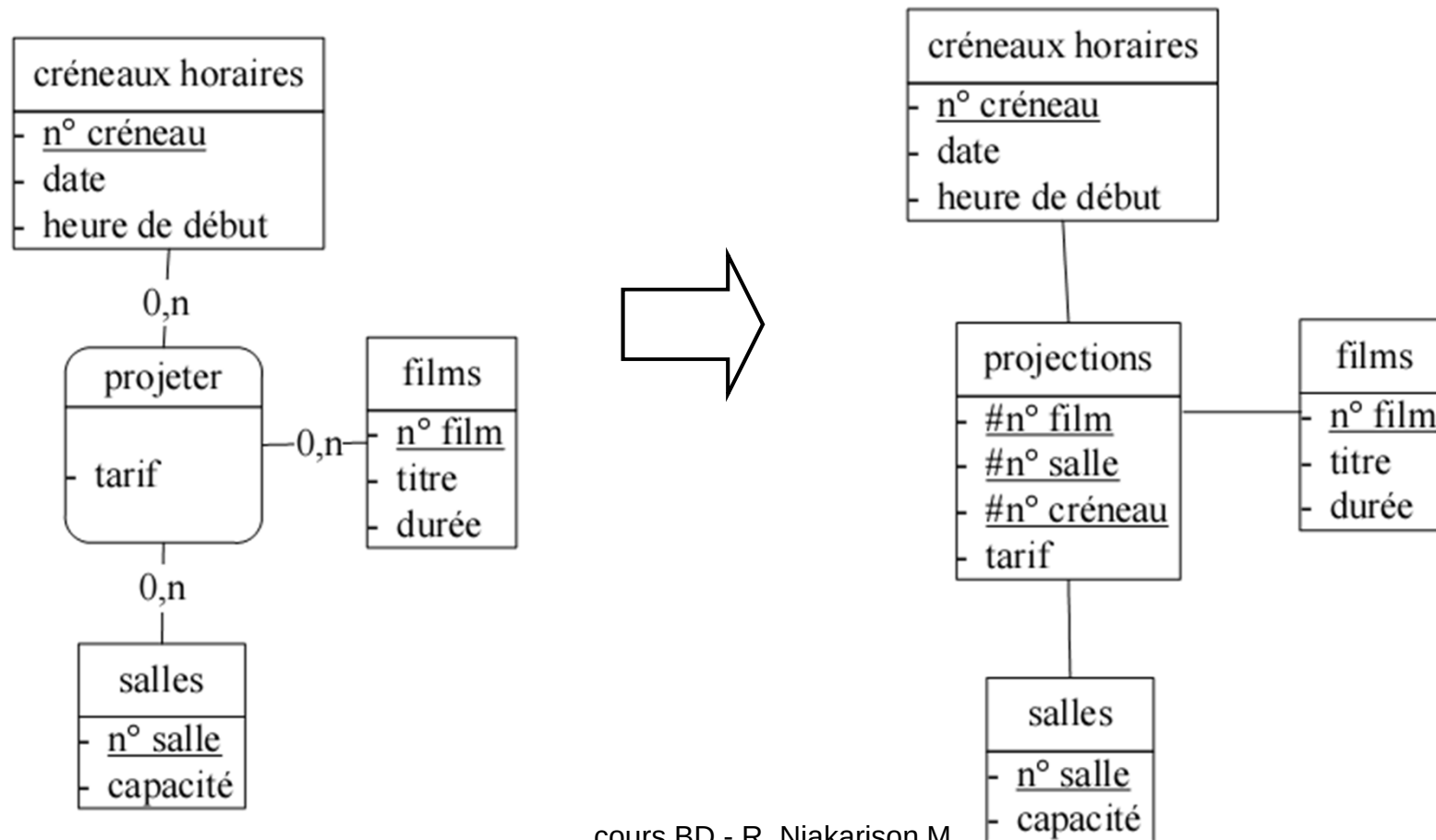
Une association binaire de type 1:1 est traduite comme une association binaire de type 1:n (particulière) sauf que la clé étrangère se voit imposer une contrainte d'unicité et une contrainte de non vacuité (fig. 1-b)

Une alternative consiste à la voir comme une association binaire de type n:m particulière, il suffit d'ajouter une contrainte d'unicité sur chacune des clés étrangères de la table de jonction. (fig. 1-c)



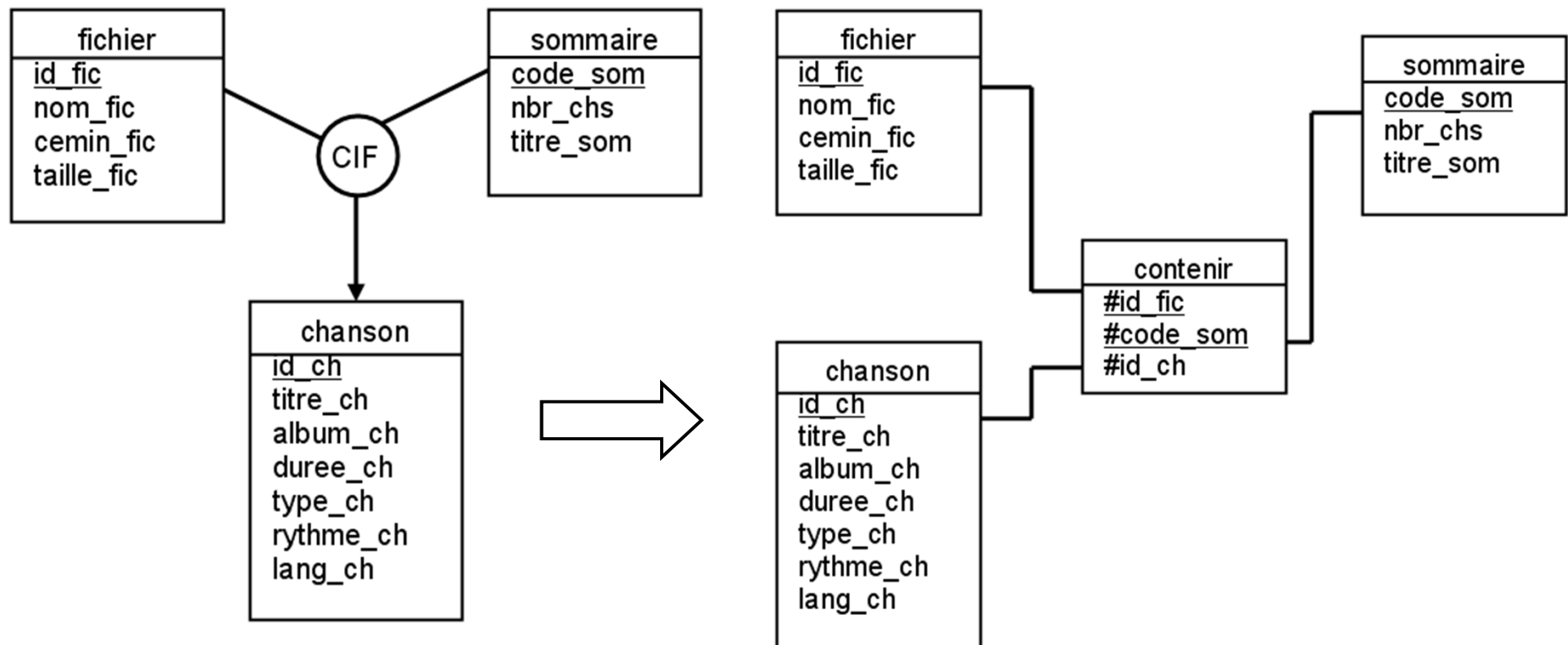
Règle N°5 a. Association non binaire pas de CIF

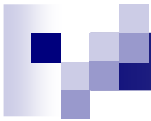
Une association non binaire est traduite par une table supplémentaire dont la clé primaire est composée d'autant de clé étrangère que d'entité en association.



Règle N°5 *b. Association non binaire CIF multiple*

Une CIF multiple (accompagné ou non) est traduite par une table supplémentaire, dont la clé primaire est formée par l'identifiant des entités sources de dépendance et l'identifiant de l'entité en dépendance devient clé étrangère de cette nouvelle table.





Contraintes d'intégrités :

Les contraintes d'intégrité sont destinées à maintenir la cohérence de la base de données. Ce sont des assertions qui doivent être vérifiées à tout moment par les données contenues dans la base. Elles rejoignent celles qui ont été discutées au niveau du modèle Entités – Associations.

Trois types de contraintes d'intégrité complètent obligatoirement la description d'un schéma relationnel :

- ❑ Contrainte de clef : chaque relation doit comporter une clef.
- ❑ Contrainte de référence (ou intégrité référentielle) : tout tuple d'une relation R qui se réfère à une relation S doit se référer à un tuple existant dans la relation S.
- ❑ Contrainte d'entité : tout attribut faisant partie de la clef d'une relation doit être non nul.



Partie3 L'algèbre relationnelle

L'algèbre relationnelle est un ensemble d'opérations agissant sur des relations et produisant des relations. L'algèbre relationnelle permet de *spécifier des interrogations* sur une base de données relationnelle.

L'algèbre relationnelle comporte :

- ❑ des opérations ensemblistes classiques : union, intersection, différence, produit cartésien,
- ❑ des opérations spécifiques au modèle relationnel : sélection, projection, jointure et renommage.



3.1. Opérations ensemblistes :

Exemple Soient les relations auteur et rédacChef sur le schéma PERSONNE(Nom, Prénom)

auteur	Nom	Prénom
	Helenen	Greco
	Matile	François
	Monnier	Claude

rédacChef	Nom	Prénom
	Ribeau	Jacques
	Monnier	Claude
	Pillet	Jacques

❖ **intersection** : $r \cap s$ contient tous les tuples qui sont à la fois de r et de s.

Les 2 opérandes r et s doivent être sur le même schéma de relation R et le résultat est une relation sur le schéma de relation R.

Si $r = \text{auteur}$ $s = \text{redacChef}$ alors on a :

$r \cap s$	Nom	Prénom
	Monnier	Claude

- 
- ❖ **Union** : $r \cup s$ contient tous les tuples de r et tous les tuples de s qui ne sont pas dans r .

Les 2 relations r et s doivent être sur le même schéma de relation R et le résultat est une relation sur le schéma de relation R .

Si $r = \text{auteur}$ $s = \text{redacChef}$ alors on a :

$r \cup s$	Nom	Prénom
	Helenen	Greco
	Matile	François
	Monnier	Claude
	Ribeau	Jacques
	Pillet	Jacques



❖ **Différence** : **r - s** contient tous les tuples de r qui ne sont pas dans s.

Les 2 relations r et s doivent être sur le même schéma de relation R et le résultat est une relation sur le schéma de relation R.

Si **r = auteur** **s = redacChef** alors on a :

r - s	Nom	Prénom
	Helenen	Greco
	Matile	François

- ❖ **Produit cartésienne** : $r \times s$ a pour schéma la juxtaposition des schémas de r et de s, et pour tuples toutes les combinaisons de tuples de r et s.

Exemple :

cru	Nom	Couleur	Région	vins	R. Prod.	Mill.	Qualité
	ch. Latour	rouge	bordeaux		bordeaux	1990	Excellente
	clos Vougeot	rouge	bourgogne		bourgogne	1990	bonne

cru $\times$ vin	Nom	Couleur	Région	R. Prod.	Mill.	Qualité
	ch. Latour	rouge	bordeaux	bordeaux	1990	Excellente
	ch. Latour	rouge	bordeaux	bourgogne	1990	bonne
	clos Vougeot	rouge	bourgogne	bordeaux	1990	Excellente
	clos Vougeot	rouge	bourgogne	bourgogne	1990	bonne



❖ *Opérations spécifiques du modèle relationnel*

Pour manipuler les tables d'une base de données relationnelles, 3 "outils" sont essentiels :

- des ciseaux qui "coupent à l'horizontale" (sélection de certaines lignes : *restriction*),
- des ciseaux qui "coupent à la verticale" (sélection de certaines colonnes : *projection*),
- de la colle (*jointure*).

1. La projection (selon une liste d'attributs) - opérateur unaire

La projection selon une liste d'attributs L se note $\pi_L(R)$ où R est une relation.

Le résultat est une table comportant seulement les attributs présents dans L . Les enregistrements redondants sont éliminés du résultat.

Soit la relation rédacChef :

rédacChef	Nom	Prénom
	Ribeu	Jacques
	Monnier	Claude
	Pillet	Jacques

$\pi_{\text{Prénom}}(\text{rédacChef})$

$\pi_{\text{Prénom}}(\text{rédacChef})$	Prénom
	Jacques
	Claude

2. La sélection (ou restriction) selon une condition - opérateur unaire

La sélection selon une condition C se note $\sigma_C(R)$ où R est une relation. Le résultat est une table de même schéma, mais comportant uniquement les tuples qui vérifient C . La condition porte sur les noms d'attributs de la relation, et les valeurs des domaines, elle est exprimée à l'aide des opérateurs comparatifs ($=, \neq, <, >, \leq, \geq$) et des opérateurs logiques ($\neg, \wedge, \vee$).

PARLER	Nom_pays	Nom_langue	Nb_locuteurs
	Bangladesh	Anglais	10
	Bangladesh	Bengali	105
	Bangladesh	Hindi-ourdou	1
	Canada	Anglais	16
	Canada	Français	7
	Etats-Unis	Anglais	250
	Etats-Unis	Espagnol	17
	Etats-Unis	Allemand	6
	Paraguay	Espagnol	3
	Paraguay	Guarani	3

SPEAK = $\sigma_{\text{Nom_langue} = \text{Anglais}}(\text{PARLER})$

SPEAK	Nom_pays	Nom_langue	Nb_locuteurs
	Bangladesh	Anglais	10
	Canada	Anglais	16
	Etats-Unis	Anglais	250

$\sigma_{(\text{Nom_langue} = \neg \text{Anglais}) \vee (\text{Nom_pays} = \text{Canada})}(\text{PARLER})$



3. Le renommage

$\rho_{x(A_1, A_2, \dots, A_n)}(R)$ où R est une relation, x son nouveau nom, A_i les nouveaux noms de ses attributs.

4. La jointure (selon une condition) - opérateur binaire

la jointure selon une condition C se note $R \bowtie_C S$ où R et S sont des relations qui *ont en commun au moins un domaine d'attribut*. L'opération de jointure permet *d'obtenir une table à partir de 2 tables* par *réunion de leurs colonnes, et concaténation de toutes leurs lignes vérifiant C* . C'est comme prendre le *produit cartésien $R \times S$ et sélectionner dans ce produit les tuples qui satisfont la condition C* . La condition est une conjonction de conditions, chacune étant de la forme $A \odot B$, $\odot$ étant un opérateur de comparaison, A étant un attribut de R , B un attribut de S , et A et B ayant tous deux le même domaine.



Exemple de jointure

cru	Nom	Couleur	Région
	ch. Latour	rouge	bordeaux
	clos Vougeot	rouge	bourgogne

vins	R. Prod.	Mill.	Couleur	Qualité
	bordeaux	1990	Rouge	Excellente
	bourgogne	1990	Rouge	bonne

cru $><$ (Couleur = Couleur) $\wedge$ (Région = R. Prod.) **vins**

Nom	Couleur	Région	R. Prod.	Mill.	Couleur	Qualité
ch. Latour	rouge	bordeaux	bordeaux	1990	rouge	Excellente
clos Vougeot	rouge	bourgogne	bourgogne	1990	rouge	bonne



Exemple de jointure

cru	Nom	Couleur	Région
	ch. Latour	rouge	bordeaux
	clos Vougeot	rouge	bourgogne

vins	R. Prod.	Mill.	Couleur	Qualité
	bordeaux	1990	Rouge	Excellente
	bourgogne	1990	Rouge	bonne

cru $\bowtie$ (Couleur = Couleur) $\wedge$ (Région = R. Prod.) **vins**

cru $\times$ vin	Nom	Couleur	Région	R. Prod.	Mill.	Qualité
	ch. Latour	rouge	bordeaux	bordeaux	1990	Excellente
	ch. Latour	rouge	bordeaux	bourgogne	1990	bonne
	clos Vougeot	rouge	bourgogne	bordeaux	1990	Excellente
	clos Vougeot	rouge	bourgogne	bourgogne	1990	bonne



Autre exemple de jointure

U	A	B	C
	1	2	3
	6	7	8
	9	7	8

V	B	C	D
	2	3	4
	2	3	5
	7	8	10

U $\bowtie_{A=D}$ V	A	U.B	U.C	V.B	V.C	D
	1	2	3	2	3	4
	1	2	3	2	3	5
	1	2	3	7	8	10
	6	7	8	7	8	10
	9	7	8	7	8	10

Equi-jointure : Si le seul opérateur de comparaison utilisé dans la condition est l'égalité alors la jointure est appelée équi-jointure.



Autre exemple de jointure

U	A	B	C
	1	2	3
	6	7	8
	9	7	8

V	B	C	D
	2	3	4
	2	3	5
	7	8	10

U	A	B	C	B	C	D
	1	2	3	2	3	4
	1	2	3	2	3	5
	1	2	3	7	8	10
	6	7	8	7	8	10
	9	7	8	7	8	10



Jointure naturelle

En cas d'équi-jointure, les colonnes jointes ont toujours un contenu identique, on peut donc en supprimer une. Dans ce cas l'opération est appelée jointure naturelle. Il faut alors que les attributs à joindre portent le même nom. La jointure est faite sur tous les attributs communs.

cru	Nom	Couleur	Région	vins	R. Prod.	Mill.	Couleur	Qualité
	ch. Latour	rouge	bordeaux		bordeaux	1990	Rouge	Excellente
	clos Vougeot	rouge	bourgogne		bourgogne	1990	Rouge	bonne

cru >< vins

Nom	Couleur	Région	Mill.	Qualité
ch. Latour	rouge	bordeaux	1990	Excellente
clos Vougeot	rouge	bourgogne	1990	bonne

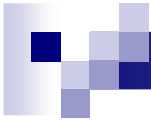


Autre exemple de jointure naturelle

U	A	B	C
	1	2	3
	6	7	8
	9	7	8

V	B	C	D
	2	3	4
	2	3	5
	7	8	10

U >< V	A	B	C	D
	1	2	3	4
	1	2	3	5
	6	7	8	10
	9	7	8	10



Partie4 Introduction à SQL

4.1 Introduction

Le langage SQL (Structured Query Language) est un standard des bases de données relationnelles, qui utilise les concepts de :

- table (relation, à part qu'une table peut contenir des doublons)
- ligne (tuple)
- colonne (attribut)
- type de donnée (domaine)

Ce langage comprend différentes parties :

- Langage de définition des données (DDL ou LDD) : définition de schémas de relations, destruction de relation, création d'index, spécification des contraintes d'intégrité, etc.
- Langage de manipulation de données (DML ou LMD) : pour poser des questions, ajouter, modifier ou supprimer des tuples.



4.2 Langage de définition de données (LDD)

4.2.1. types de données

Les principaux types des valeurs des attributs sont les suivants :

- chaînes de caractères : CHAR(n), VARCHAR(n)
- nombres : entiers (INT, SMALLINT), nombres à virgule de taille fixée (NUMERIC(3, 1) pour des nombres de 3 chiffres, dont 1 après la virgule), réels (REAL, FLOAT(n))
- date : DATE année (à 4 chiffres), mois, jour du mois
- horaire : TIME heures, minutes, secondes



4.2.2. Définition du schéma d'une table

```
CREATE TABLE nom_table(  
    A1 D1,  
    A2 D2,  
    .....,  
    An Dn,  
    contrainte d'intégrité 1,  
    .....,  
    contrainte d'intégrité k)
```

Où A_i est le i ème attribut, D_i son type. On peut indiquer les contraintes avec l'attribut, ou après. Les principales contraintes d'intégrité sont : PRIMARY KEY et CHECK.

PRIMARY KEY($A_1, \dots, A_n$) indique que les attributs $A_1, \dots, A_n$ forment une clef primaire.

CHECK(C) indique que la condition C doit être satisfaite par chaque ligne de la table.

NOT NULL, interdit à un attribut de ne pas avoir de valeur nulle.



Exemple de définition du schéma d'une table

```
CREATE TABLE Agence (  
  Nom_agence CHAR(15) NOT NULL,  
  Ville_agence CHAR(30),  
  Capital INTEGER,  
  PRIMARY KEY (Nom_agence),  
  CHECK (Capital >= 0)  
)
```

```
CREATE TABLE Client (  
  Nom_client CHAR(20) NOT NULL,  
  Rue_client CHAR(30),  
  Ville_client CHAR(30),  
  PRIMARY KEY (Nom_client)  
)
```

```
CREATE TABLE Depot (  
  Nom_client CHAR(20) NOT NULL,  
  Num_compte CHAR(10) NOT NULL,  
  PRIMARY KEY (  
    Nom_client,  
    Num_compte  
  )  
)
```

```
CREATE TABLE Compte (  
  Num_compte CHAR(10) NOT NULL  
  PRIMARY KEY ,  
  Nom_agence CHAR(15),  
  Solde INTEGER,  
  CHECK (Solde >= 0)  
)
```



Remarques

- *La condition du CHECK peut être une requête*, par exemple pour la relation Compte :

CHECK(Nom_agence IN (SELECT Nom_agence FROM Agence))

- *Il est possible de donner une valeur par défaut à un attribut*, par exemple pour l'attribut Anniversaire d'une table de clients : Anniversaire
DATE DEFAULT DATE '0000-00-00'



4.2.3 Modification des objets de la base

La commande générale est ALTER. Et on indique le type d'objet à modifier (table, attributs,...)

- Pour ajouter l'attribut A de type D au schéma d'une table T, on écrira :

ALTER TABLE T ADD A D

Lorsqu'on ajoute un attribut à une table existante, chaque ligne reçoit la valeur Null pour ce nouvel attribut.

- Pour supprimer l'attribut A du schéma d'une table T, on écrira :

- *ALTER TABLE T DROP A*

Suppression des objets de la base

La commande générale est DROP. Et on indique le type d'objet à modifier (table, vue, index...)

- Pour supprimer une table T, on écrira : *DROP TABLE T*



4.2.4 Modification de **DONNÉES**

Une fois le schéma de la base spécifié, on peut remplir les tables...

1. Ajout de données dans une table

INSERT INTO T VALUES (a₁, a₂, ..., a_n) ajoute dans la table T la ligne de valeurs (a₁, a₂, ..., a_n), les a_i étant, dans l'ordre, les valeurs des attributs de T Exemple : "Ajouter un compte de n°A-973 et de solde 2000 à Perryridge"

```
INSERT INTO Compte VALUES ("Perryridge", "A-973", 2000)
```

2. Suppression de données dans une table

DELETE FROM T WHERE C supprime de la table T les lignes qui satisfont la condition C Exemple : "Supprimer tous les prêts dont le montant se situe entre 10000 et 13000"

```
DELETE FROM Pret WHERE Montant BETWEEN 10000 AND 13000
```

3. Mises a jour des données d'une table

UPDATE T SET A = V WHERE C affecte à l'attribut A de la table T la valeur V pour les lignes vérifiant C.



4.3 Contraintes d'intégrité

Les contraintes sont automatiquement vérifiées à chaque modification des données de la base. Si l'une des contraintes n'est pas satisfaite, la modification n'est pas effectuée :

4.3.1. Les clefs (primary key, vue précédemment)

4.3.2. L'intégrité référentielle

Lorsqu'une valeur d'une ligne d'une table T1 nécessite la présence d'une valeur d'une ligne dans une table T2. Par exemple, on ne souhaite pas qu'il y ait dans la table Compte un compte dans l'agence "Blois-Vienne" s'il n'existe pas dans la table Agence d'agence appelée "Blois-Vienne". On dit que l'attribut Nom_agence dans la table Compte est une "clef externe".



On l'exprime de la façon suivante en SQL :

```
CREATE TABLE Compte(  
    Num_compte CHAR(10) NOT NULL,  
    Nom_agence CHAR(15),  
    Solde INTEGER,  
    PRIMARY KEY (Num_compte),  
    FOREIGN KEY (Nom_agence) REFERENCES Agence,  
    CHECK (Solde >= 0)  
)
```

La clef externe doit référencer la clef primaire de l'autre table.

On peut aussi écrire directement :

```
Nom_agence CHAR(15) REFERENCES Agence
```



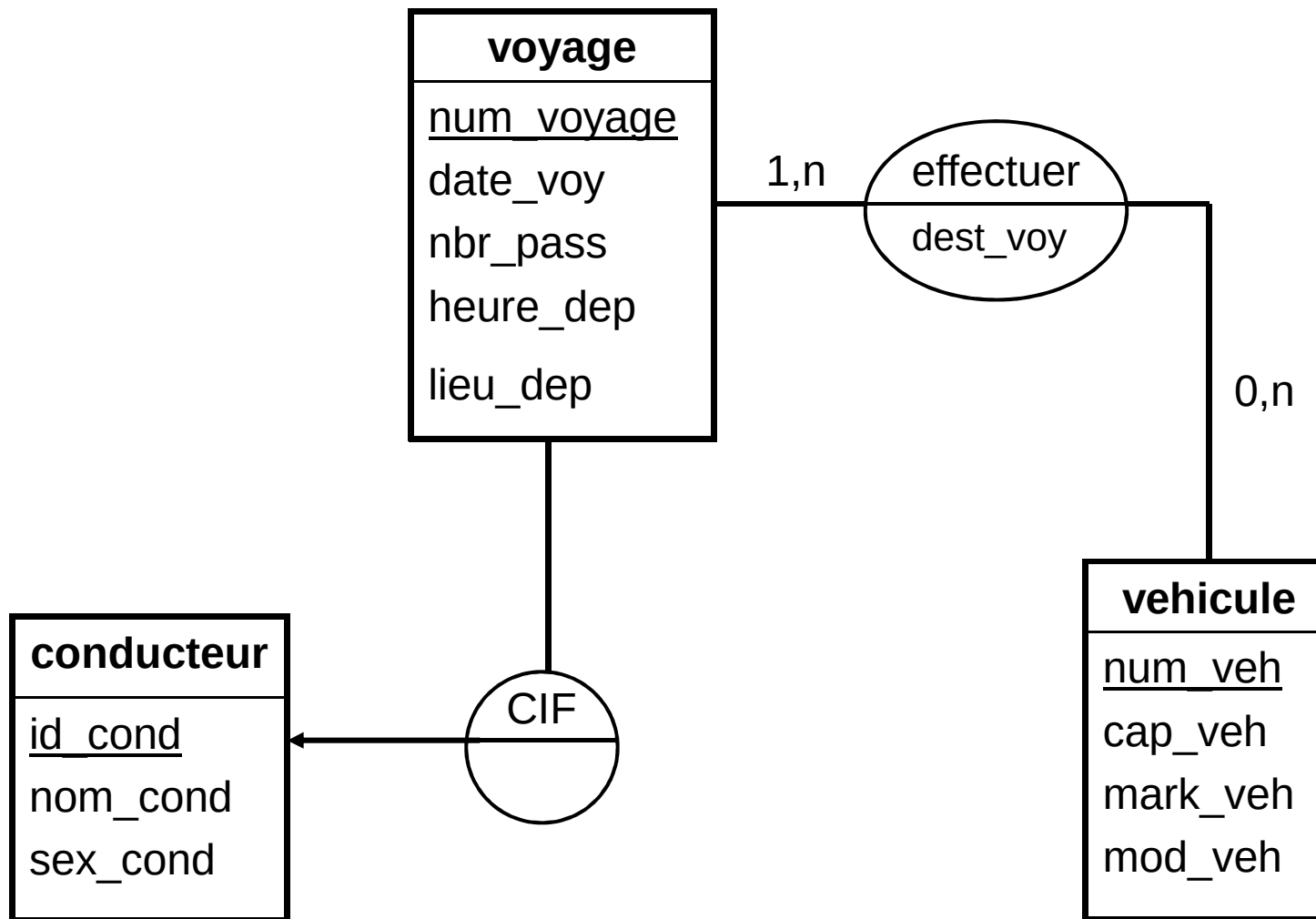


Exercice d'application :

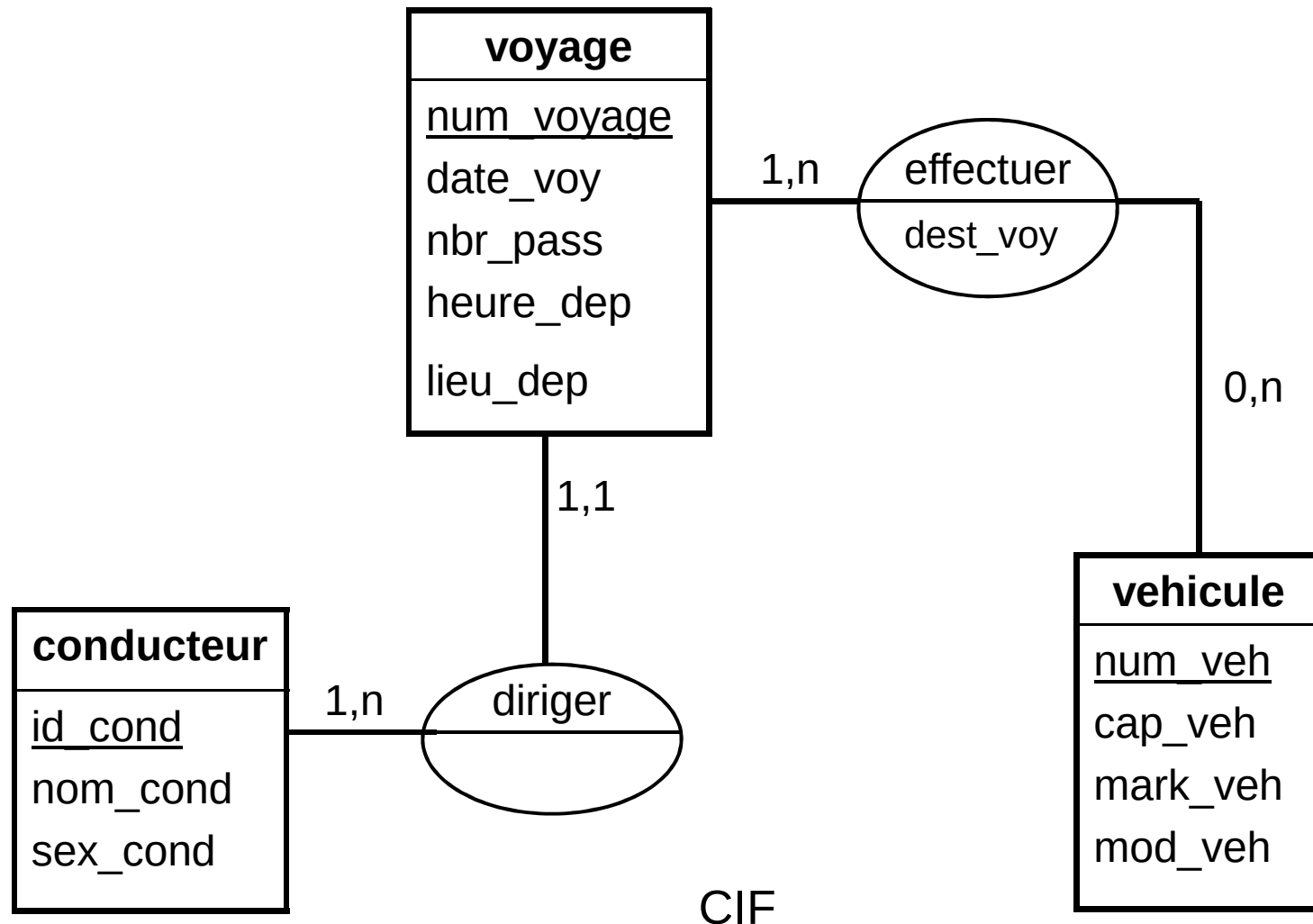
Un magasin de sport a besoin de stocker ses informations principales dans une base de données. Il s'agit de stocker toutes les informations relatives à ses produits, ses clients et ses fournisseurs. Il souhaite que l'organisation soit facilitée par le regroupement des produits en différentes catégories. Et il souhaite à partir de là pouvoir facilement consulter ses stocks, la liste de ses meilleurs clients, le top 10 de ses produits vendus, ou la facture d'un client donné...

Créez les MCD correspondant.

finalement on obtient



Traduction du GDF en schéma ER





Exercice d'application :

II. Gestion des stages

L'ESP veut organiser des stages à l'intention des étudiants de deuxième année (code_etu, nom&prenom, n_insee). Il fait appel à des entreprises (code_ent, adresse_ent) qui proposent des stages. Les stages sont caractérisés par :

- un code stage (n_stage)
- un thème d'étude : informatique,..... (theme)

Les étudiants indiquent les stages qu'ils souhaitent suivre en priorité en leur attribuant un numéro (n_priorite). *Le chef de département décide alors quel stage sera suivi par chaque étudiant.*

Un enseignant de l'ESP (code_resp) peut avoir la responsabilité de plusieurs stagiaires. Cet enseignant sera disponible à un jour donné de la semaine et à un horaire précis afin de donner tout renseignement concernant un ou plusieurs stages.

Faire le schéma Entité-Association de la Gestion des stages



Gestion de la scolarité

Lors de son inscription en début d'année scolaire, chaque étudiant remplit une fiche sur laquelle il indique certains renseignements comme :

- le numéro d'identification nationale (n_insee)
- le nom et le prénom (nom, prenom)
- l'adresse (adresse)
- la liste des unités de valeurs qu'il devra suivre en cours d'année (8 au maximum dans une liste de 15 possibles)

Un code étudiant lui est automatiquement attribué (code_etu). Une unité de valeur est caractérisée par un code (code_uv) et un intitulé (int_uv). Par exemple l'UV 3 est l'UV Electronique numérique,

Chaque UV est placée sous la responsabilité d'un enseignant ayant un code (code_ens), un nom (nom_ens), un numéro de bureau (bureau_ens) et un numéro de téléphone (tel_ens).

Cet enseignant sera disponible à un jour donné de la semaine (j_sem) et à un horaire précis (h_rens) afin de donner tout renseignement concernant cette UV.

Faire le schéma Entité-Association de la gestion de la scolarité



Faire le schéma Entité-Association de la gestion de la scolarité

Rappels : Commencer par recenser les propriétés

Etablir le dictionnaire de données

Faire le graphe de dépendance fonctionnelle

Traduire le graphe de DF en MCD

Inscrire les cardinalités

EQUIPE

<u>nume</u>	nome
BD1	SGBD Relationnel
BD2	SGBD Objet
BD3	SGBD Repartis
SIG1	Systeme Informations
IA1	Intelligence artificielle
RDP1	Réseaux Petri

PROJET

<u>nump</u>	designation	nequipe#	pbudg
ESPRIT1	Data Warehouse	BD3	90050
ESPRIT2	Data Mining	BD1	134500
ESPRIT3	Neural Network	IA1	340050
ERASMUS1	Data and Web	BD2	78000
ERASMUS2	Web Object	BD2	90050
EFFORT1	Oracle Ware	BD3	96070

CHERCHEUR

<u>numch</u>	nomch
4	Durand
7	Dupond
9	Legrand
3	Song
8	Spaccapiettra
19	Chen
34	Teorey
20	Payrissat

AFFECTATION

<u>numaffeq#</u> ,	<u>numaffch#</u>
BD1	8
SIG1	9
BD1	9
BD3	8
BD2	19
BD1	7
BD2	3
IA1	20
BD2	8
BD3	3
BD3	19



Opérateurs unaires : Requêtes mono - tables

- 1/ Numéro des équipes auxquelles sont associées au moins un projet
- 2/ Numéro des équipes ayant un budget de projet > 91000 Francs
- 3/ Projets de l'équipe 'BD3' dont les budgets ne dépassent pas 91000 Francs

Opérateurs binaires et plus : Requêtes multi - tables

- 4/ Noms des chercheurs qui sont affectés à l'équipe de code 'BD2' ?
- 5/ Noms des chercheurs qui sont affectés à l'équipe dont le nom est 'SGBD Objet'?

Opérateurs ensemblistes

- 6/ Codes des chercheurs qui ne sont pas affectés à une équipe ?
- 7/ Noms des chercheurs qui ne sont pas affectés à une équipe ?
- 8/ Codes des chercheurs qui sont affectés aux équipes 'BD1' et 'BD3'
- 9/ Noms des chercheurs qui sont affectés aux équipes 'BD1' et 'BD3'
- 10/ Codes des chercheurs qui sont affectés aux mêmes équipes que le chercheur 19



SQL

Introduction

Le langage SQL (Structured Query Language) est un standard (norme ISO) des bases de données relationnelles, qui utilise les concepts de :

- table (relation, à part qu'une table peut contenir des doublons)
- ligne (tuple)
- colonne (attribut)
- type de donnée (domaine)

Ce langage comprend différentes parties :

- ☐ Langage de définition des données (DDL ou LDD) : définition de schémas de relations, destruction de relation, création d'index, spécification des contraintes d'intégrité, etc.
- ☐ Langage de manipulation de données (DML ou LMD) : pour poser des questions, ajouter, modifier ou supprimer des tuples.